



Criação de Agentes Autônomos

Autoria: Rodrigo Fernandes

Introdução

Ao longo deste módulo, eu construí um agente autônomo completo partindo do zero e evoluindo progressivamente até chegar em um sistema que não apenas executa tarefas, mas aprende, decide e melhora continuamente com base em experiência.

O ponto de partida foi entender que agentes autônomos não são automações tradicionais. Sistemas clássicos seguem fluxos determinísticos baseados em condições explícitas. Já agentes operam em um ambiente incerto, onde precisam interpretar contexto, tomar decisões e lidar com situações não previstas.

Essa diferença muda completamente a forma como projetamos software.

Fundamentos dos agentes e o Agent Loop

Um agente opera em ciclo contínuo. Esse ciclo é composto por percepção, raciocínio, ação e feedback. Não é um fluxo linear, mas um sistema iterativo de decisão.

O agente observa o estado atual, decide o próximo passo, executa e avalia o resultado. Se o objetivo ainda não foi atingido, ele repete o ciclo.

Esse modelo traz um risco importante: loops infinitos. Diferente de automações que falham rapidamente, um agente mal projetado pode continuar executando indefinidamente, consumindo recursos sem gerar valor. Isso não é falha do modelo, mas da arquitetura do loop.

Outro ponto crítico é a percepção. O agente não deve trabalhar apenas com um input isolado. Ele precisa construir um estado completo, incluindo contexto, histórico e sinais de risco. Sem isso, qualquer decisão será baseada em informação incompleta.

Contratos como base da arquitetura

Para transformar esse comportamento em um sistema real, eu introduzi o conceito de contratos. O agente não é definido por código, mas por arquivos que descrevem seu comportamento.

Esses contratos definem identidade, objetivo, regras, ferramentas, memória e formato de saída. O runtime apenas executa o que está definido nesses arquivos.

Essa abordagem permite separar claramente responsabilidade:

- O contrato define o comportamento
- O runtime executa
- A LLM decide dentro dos limites

Isso garante controle, previsibilidade e governança.

Runtime e execução

O runtime é o motor que orquestra o agente. Ele lê os contratos, executa o loop e integra com a LLM.

Ele é completamente genérico. Não sabe o que o agente faz, apenas como executar o que está definido.

Essa separação permite criar múltiplos agentes com comportamentos diferentes usando o mesmo motor.

Observabilidade e trace

Logs tradicionais não são suficientes para agentes. Eles não explicam por que uma decisão foi tomada.

Por isso, introduzi o conceito de trace. O trace registra toda a execução do agente, incluindo cada decisão, tempo, consumo de tokens e etapas percorridas.

Com isso, é possível responder perguntas fundamentais:

- Por que o agente escolheu essa ação?
- Quantas decisões foram tomadas?

- Qual foi o custo da execução?

Sem observabilidade, não existe controle.

Tipos de agentes

Nem todo agente é igual. Existem diferentes modos de operação, definidos no contrato:

- Task-based: executa tarefas diretas
- Interativo: faz perguntas antes de agir
- Goal-oriented: decompõe objetivos
- Autônomo: reage a eventos com limites rígidos

O runtime é o mesmo, mas o comportamento muda completamente com base no tipo.

Arquiteturas cognitivas

Na sequência, eu aprofundei na forma como o agente pensa. Não basta executar, é necessário raciocinar corretamente.

Introduzi o conceito de arquitetura cognitiva, que define como o agente organiza seu raciocínio.

Três principais arquiteturas foram trabalhadas:

- React: pensa e age passo a passo
- Plan-and-Execute: planeja tudo antes de executar
- Reflection: aprende com erros

Cada uma tem vantagens e desvantagens. O React é flexível, mas caro em tokens. O Plan-and-Execute é eficiente, mas menos adaptável.

A escolha não deve ser baseada em opinião, mas em dados.

Evals e benchmarks

Para sair do achismo, introduzi o conceito de eval. Avaliação automatizada baseada em três elementos:

- Dataset de cenários
- Métricas
- Limites de aprovação

O agente é executado em múltiplos cenários e seu comportamento é medido.

Isso permite responder perguntas como:

- Ele funciona apenas em um caso ou em vários?
- Ele é eficiente?
- Ele toma boas decisões?

Sem eval, não existe engenharia, apenas tentativa e erro.

Integração com dados reais

Até então, o agente operava com dados simulados. Isso é útil para aprendizado, mas inviável em produção.

O uso de dados falsos leva a diagnósticos incorretos, falsos positivos e perda de confiança.

A solução foi introduzir adapters, que conectam o agente a sistemas reais como APIs e bancos de dados.

Essa integração exige cuidados adicionais, especialmente com segurança.

Segurança e adapters

Ao trabalhar com banco de dados, o risco é significativamente maior. Diferente de APIs, bancos possuem estado e dados críticos.

Por isso, defini três regras fundamentais:

- Apenas leitura
- Queries parametrizadas

- Validação rigorosa

Essas práticas evitam problemas como SQL Injection e operações destrutivas.

Avaliação de escolha de ferramentas

Além de avaliar o agente como um todo, também é necessário avaliar decisões específicas, como a escolha de ferramentas.

Introduzi métricas como:

- Tool Selection Accuracy
- Argument Accuracy
- Wrong Tool Rate

Isso permite identificar se o agente está tomando decisões corretas em cada etapa.

Introdução à memória

Até esse ponto, o agente não lembrava de nada entre execuções.

Isso significa repetir trabalho, gastar tokens desnecessários e perder eficiência.

A solução foi introduzir quatro tipos de memória:

- Curta: estado da execução atual
- Longa: conhecimento persistente
- Episódica: histórico de execuções
- Contextual: recuperação por similaridade

Essa estrutura permite que o agente evolua com o tempo.

Embeddings e busca por significado

A memória contextual é baseada em embeddings, que transformam texto em representações numéricas.

Isso permite buscar informações por significado, e não apenas por palavras exatas.

Com isso, o agente consegue recuperar conhecimento relevante mesmo quando a terminologia muda.

Reflexão evolutiva

Além de lembrar, o agente passa a aprender.

A reflexão analisa execuções passadas, extrai lições e melhora decisões futuras.

Isso transforma o agente em um sistema adaptativo.

Avaliação do impacto da memória

A introdução de memória não garante melhoria. Ela pode ajudar, ser irrelevante ou prejudicar.

Por isso, medi o impacto com seis métricas:

- Retrieval Precision
- Retrieval Recall
- Memory Utilization
- Hallucination from Memory
- Decision Improvement
- Lesson Quality

A métrica mais importante é o Decision Improvement, que compara o desempenho com e sem memória.

Governança e arquitetura

A principal conclusão do módulo é que memória não é uma funcionalidade opcional. Ela é parte da arquitetura.

Ela altera o ciclo do agente, adiciona novas responsabilidades e exige governança.

O comportamento do agente não depende apenas do modelo, mas dos contratos, regras, adapters e métricas.

Conclusão

Ao final do módulo, eu construí um agente completo que:

- Possui ciclo de decisão estruturado
- É governado por contratos
- Possui observabilidade
- Utiliza dados reais
- Avalia seu próprio desempenho
- Aprende com experiência

A evolução não está no modelo, mas na arquitetura.

O agente deixa de ser uma ferramenta e passa a se comportar como um sistema inteligente, capaz de tomar decisões cada vez melhores com o tempo.

Módulo 1: Introdução aos Agentes Autônomos

Capítulo 1: Arquitetura de Agents

Seja muito bem-vindo ao módulo 4, onde eu passo a tratar sobre agentes autônomos. Ao longo desta unidade, eu vou te conduzir por uma mudança importante de mentalidade sobre como construímos software, especialmente quando integramos inteligência artificial nos nossos sistemas.

Eu venho de uma trajetória de mais de duas décadas como engenheiro de software, e nos últimos anos tenho direcionado meu trabalho para um campo que vem ganhando força no mercado: a engenharia de software aplicada à inteligência artificial. Esse movimento é frequentemente chamado de Applied AI Engineering.

É importante deixar claro desde o início que o foco aqui não é formar cientistas de dados. Não vamos entrar em treinamento de modelos, fine-tuning ou construção de LLMs. O objetivo é outro: construir sistemas reais que utilizam inteligência artificial como ferramenta para tomada de decisão.

Isso significa trabalhar com execução de tarefas, integração entre sistemas, automação de fluxos e, principalmente, aplicar princípios sólidos de engenharia de software nesse contexto.

O Problema dos Sistemas Tradicionais

Antes de aprofundar o conceito de agentes, eu gosto de propor um cenário bastante comum.

Imagine que você realizou o deploy de um sistema considerado estável. Ele passou por testes, revisões, está em produção e funcionando conforme esperado. No entanto, em algum momento, surge um fluxo não previsto, uma entrada inesperada, algo que não estava no escopo original.

O que acontece?

O sistema simplesmente para de responder corretamente. Ele não necessariamente quebra, não lança exceção, não gera logs úteis. Ele apenas deixa de funcionar como deveria.

Você verifica logs, métricas, dashboards, e não encontra evidência clara do problema. O sistema está operacional do ponto de vista de infraestrutura, mas funcionalmente travado.

Esse tipo de situação revela uma limitação fundamental dos sistemas tradicionais: eles são construídos para seguir um roteiro. São baseados em lógica determinística, em decisões binárias, em estruturas como if e else.

O problema é que o mundo real não funciona dessa forma.

Execução versus Decisão

Existe uma diferença fundamental que precisamos compreender: executar não é o mesmo que decidir.

Grande parte dos sistemas que construímos hoje são orientados à execução. Eles seguem fluxos pré-definidos, executam tarefas sequenciais e respondem a entradas específicas.

No entanto, muitas atividades do nosso dia a dia como desenvolvedores não são meramente executáveis. Elas envolvem julgamento, contexto e tomada de decisão.

Considere alguns exemplos:

Revisão de pull requests, onde é necessário equilibrar risco e qualidade

Triagem de incidentes em produção, decidindo o que investigar primeiro

Avaliação de mudanças arquiteturais e seus possíveis impactos

Essas tarefas não são mecânicas. Elas exigem análise contextual, avaliação de consequências e escolha entre alternativas.

É exatamente nesse ponto que surge a necessidade de algo além de automação.

O Limite das Automações

Durante décadas, evoluímos na construção de sistemas automatizados. Esses sistemas são excelentes para executar tarefas bem definidas, com regras claras e fluxos previsíveis.

No entanto, automações possuem uma limitação estrutural: elas não tomam decisões. Elas apenas seguem instruções.

Mesmo quando utilizamos LLMs em aplicações modernas, muitas vezes ainda estamos operando nesse modelo. Enviamos um prompt, recebemos uma resposta. Entrada, processamento e saída.

Esse padrão é essencialmente reativo.

Quando você interage com um modelo de linguagem dessa forma, você está consumindo uma API. O modelo responde ao que foi solicitado, mas não toma iniciativa, não define ações e não evolui o contexto por conta própria.

Isso está muito distante do conceito de um agente autônomo.

O Que é um Agente Autônomo

Um agente autônomo é um sistema que opera de maneira diferente.

Em vez de reagir a prompts isolados, ele trabalha sobre situações. Ele analisa contexto, toma decisões, executa ações, avalia resultados e decide o próximo passo.

Tudo isso sem intervenção humana contínua.

Uma definição formal que eu utilizo é a seguinte:

Um agente é um sistema que percebe o ambiente, toma decisões com base em objetivos e executa ações de forma autônoma.

De forma mais técnica, podemos decompor um agente em alguns componentes fundamentais:

Percepção do ambiente

Políticas de decisão

Ações

Feedback

O ponto central aqui é que o agente opera em loop. Ele não executa uma única vez e encerra. Ele continua avaliando o estado do sistema e ajustando seu comportamento conforme necessário.

A Importância do Objetivo

Existe um elemento que define completamente o comportamento de um agente: o objetivo.

Sem um objetivo explícito, não existe agente. O que existe é apenas uma automação.

Uma automação executa passos. Um agente persegue um objetivo.

Essa diferença é sutil na teoria, mas extremamente significativa na prática. Enquanto uma automação segue um roteiro fixo, um agente decide quais passos

seguir ao longo do caminho, com base no contexto e no objetivo que precisa atingir.

O objetivo é o que orienta todas as decisões do agente. É o critério que permite avaliar se uma ação foi bem-sucedida ou não.

Skills não são Agentes

Outro ponto importante é a distinção entre agentes e skills.

Skills são habilidades. São capacidades específicas que um agente pode utilizar para executar ações.

Por exemplo, consultar logs, analisar métricas, enviar notificações ou interagir com APIs são exemplos de habilidades.

O agente, por sua vez, é o sistema que utiliza essas habilidades para atingir um objetivo.

Confundir esses dois conceitos é comum, especialmente quando começamos a trabalhar com ferramentas que permitem definir capacidades para LLMs. No entanto, é essencial entender que skills são ferramentas, enquanto o agente é o tomador de decisão.

Um Exemplo Prático

Considere um cenário de monitoramento de produção.

Em um sistema tradicional, ao detectar um problema de latência, uma automação pode enviar um alerta. A partir daí, um humano assume a responsabilidade de investigar.

Com um agente autônomo, o comportamento é diferente.

Ao receber o alerta, o agente pode:

Analisar métricas relacionadas

Consultar logs do período afetado

Correlacionar com deploys recentes

Avaliar padrões históricos

Com base nessas informações, ele pode decidir:

Abrir um incidente

Executar um rollback

Notificar a equipe com um diagnóstico completo

Tudo isso acontece de forma autônoma, guiado por contexto e objetivo.

O Agent Loop

O mecanismo que viabiliza esse comportamento é o que chamamos de Agent Loop.

Esse ciclo é composto por etapas fundamentais:

Perceber o ambiente

Raciocinar sobre o contexto

Agir

Avaliar o resultado

Repetir o processo

Esse loop continua até que o objetivo seja atingido ou até que um critério de parada seja alcançado.

A existência de um critério de parada é essencial. Sem ele, o agente pode entrar em execução contínua, consumindo recursos de forma descontrolada.

O Papel do Modelo de Linguagem

Um ponto que gera muita confusão é o papel do modelo de linguagem dentro desse sistema.

O modelo de linguagem não é o agente.

Ele é apenas o motor de raciocínio.

Quem decide é o agente.

A qualidade do sistema não depende exclusivamente do modelo utilizado, mas sim de como o loop de decisão foi projetado, quais são os controles implementados e como os limites foram definidos.

Engenharia de Software Aplicada a Agentes

Para que agentes autônomos sejam viáveis em produção, é necessário aplicar princípios clássicos de engenharia de software.

Isso inclui:

Definição de limites

Observabilidade

Controle de execução

Tratamento de erros

Prevenção de loops infinitos

Sem esses elementos, o sistema se torna imprevisível e inviável.

Spec Driven Agents

Uma abordagem que utilizamos para estruturar esses sistemas é chamada de Spec Driven Agents.

Nessa abordagem, o comportamento do agente é definido por especificações, geralmente escritas em Markdown.

Esses arquivos funcionam como contratos. Eles descrevem:

Identidade do agente

Objetivos

Capacidades

Limites

Regras de decisão

O uso de Markdown permite versionamento, legibilidade e integração com ferramentas de engenharia.

Isso transforma o desenvolvimento de agentes em um processo estruturado, e não em tentativa e erro.

O Que Será Construído

Ao longo desta unidade, o objetivo é construir um sistema completo.

Você não apenas entenderá o conceito de agentes, mas também desenvolverá um mini framework com:

Execução via linha de comando

Logs estruturados

Controle de execução

Proteções contra comportamento indesejado

Além disso, você será incentivado a aplicar esse conhecimento em um problema real, construindo um agente que possa fazer parte do seu portfólio.

Encerramento da Aula

Ao final desta aula, eu quero que você carregue uma ideia fundamental: agentes autônomos representam uma mudança na forma como pensamos software.

Não se trata apenas de automatizar tarefas, mas de construir sistemas capazes de decidir.

Essa mudança impacta diretamente a forma como projetamos sistemas, definimos responsabilidades e pensamos arquitetura.

A partir daqui, a tendência é que você comece a enxergar oportunidades de aplicar agentes em diversos contextos do seu dia a dia.

Capítulo 2: Agents Loop

Dando continuidade ao nosso estudo sobre agentes autônomos, eu passo agora a detalhar o que considero o núcleo desse tipo de sistema: o Agent Loop.

Na aula anterior, eu apresentei o conceito de agentes e destaquei que eles operam em ciclo. Agora, o objetivo é aprofundar cada uma das fases desse ciclo e mostrar, na prática, como esse comportamento difere completamente de uma automação tradicional.

O Agent Loop é composto por quatro etapas principais:

- Percepção
- Raciocínio
- Ação
- Feedback

No entanto, é fundamental entender que isso não é um fluxo linear. Não estamos lidando com um pipeline fixo. Estamos lidando com um sistema de decisão contínuo.

Enquanto o objetivo do agente não for atingido, esse ciclo se repete indefinidamente, sempre com base no estado atual do sistema.

O Problema dos Loops Descontrolados

Antes de aprofundar cada etapa, eu gosto de trazer um problema bastante comum na engenharia de software: loops descontrolados.

É relativamente frequente encontrarmos sistemas que entram em execução contínua sem progresso real. Eles não quebram, não lançam erros explícitos, mas continuam consumindo recursos de forma indefinida.

Alguns exemplos típicos incluem:

- Jobs que nunca finalizam
- Retries sem limite em filas
- Pipelines travados em estados intermediários

Esse tipo de comportamento não é um erro isolado. Ele é, na verdade, um sintoma claro de falta de controle.

Quando trazemos esse cenário para agentes autônomos, o problema se torna ainda mais crítico. Um agente em loop infinito não apenas continua executando, ele continua decidindo.

E se ele não possui critérios claros de parada, ele simplesmente não sabe que deveria interromper sua execução.

Esse não é um problema do modelo de linguagem. É um problema de engenharia, mais especificamente do desenho do loop.

Percepção: Construção de Estado

A primeira etapa do Agent Loop é a percepção.

Um erro comum é tratar percepção como simples leitura de um input. Na prática, percepção é a construção de um estado.

Quando um agente recebe um sinal, como um alerta de latência, essa informação isolada não é suficiente para tomada de decisão.

É necessário enriquecer esse dado com contexto, como por exemplo:

- Qual serviço está sendo afetado
- Quando o problema começou
- Se houve deploy recente

- Se esse comportamento já ocorreu anteriormente

Esse processo é equivalente ao que fazemos naturalmente como engenheiros ao investigar um incidente.

O agente precisa reproduzir esse comportamento. Ele precisa coletar, correlacionar e estruturar informações para formar um estado consistente.

Além disso, é nessa fase que identificamos sinais de risco, como:

- Tentativas repetidas
- Falta de informação
- Execuções sem permissão
- Crescimento anormal de custo

Sem uma percepção adequada, qualquer decisão será tomada no escuro. E isso inevitavelmente leva a resultados inconsistentes.

A lógica é simples: sem contexto correto, não existe decisão correta.

Raciocínio: Da Análise à Decisão Executável

Uma vez que o estado foi construído, o próximo passo é o raciocínio.

Aqui é importante desfazer um equívoco comum: raciocinar não é apenas pensar bem. Raciocinar, no contexto de agentes, é produzir uma decisão executável.

Considere um exemplo simples: identificar que um problema pode estar relacionado ao banco de dados, à rede ou ao cache.

Essa análise, por si só, não resolve o problema. Ela não leva a uma ação concreta.

O agente precisa ir além. Ele precisa transformar análise em decisão.

Uma decisão executável seria, por exemplo:

Consultar métricas de latência de um serviço específico na última hora para verificar degradação após um deploy.

Esse tipo de decisão move o sistema para frente. Ele define claramente o próximo passo.

Outro ponto essencial aqui é o critério de sucesso. Toda decisão precisa estar associada a um resultado esperado.

Sem esse critério, o agente não consegue avaliar se deve continuar, replanejar ou encerrar.

Ação: Execução Controlada

A terceira etapa do loop é a ação.

É fundamental entender que ação não significa liberdade irrestrita. Pelo contrário, ação deve ser controlada.

O agente deve operar estritamente dentro das capacidades que foram definidas para ele.

Para ilustrar, imagine um desenvolvedor com acesso irrestrito à produção, podendo executar qualquer comando a qualquer momento. Esse cenário é claramente inseguro.

O mesmo princípio se aplica aos agentes.

Cada ação deve ser:

- Limitada
- Autorizada
- Observável

Se uma ação não está explicitamente disponível, ela simplesmente não deve existir para o agente.

Esse controle é o que torna o sistema previsível e seguro.

Feedback: Avaliação e Aprendizado

Após executar uma ação, o agente entra na fase de feedback.

Aqui, a pergunta central é: o que aconteceu funcionou?

Esse ponto é frequentemente negligenciado em sistemas tradicionais. Muitas vezes, executamos ações sem validar corretamente seus resultados.

Isso gera um problema sério: erros passam a fazer parte do processo sem serem corrigidos.

No contexto de agentes, isso é inaceitável.

Cada resultado precisa ser classificado. Por exemplo:

- Sucesso
- Falha
- Resultado incompleto

Com base nessa classificação, o agente decide o próximo passo:

- Continuar
- Replanejar
- Solicitar mais informações
- Encerrar a execução

Além disso, é essencial atualizar o estado do agente. Essa atualização representa a memória do sistema.

Sem memória, o agente não aprende com suas ações e tende a repetir os mesmos erros indefinidamente.

A Importância do Critério de Parada

Mesmo com todas as etapas bem definidas, um agente ainda pode falhar se não houver um critério de parada.

Esse é um dos pontos mais críticos no desenho de sistemas autônomos.

Se não houver uma condição explícita que indique quando o agente deve encerrar sua execução, ele continuará operando indefinidamente.

Isso leva a problemas como:

- Consumo excessivo de recursos
- Aumento de custos
- Amplificação de falhas

Definir limites claros não é opcional. É uma responsabilidade essencial da engenharia.

Exemplo Completo do Loop

Para consolidar o entendimento, vamos retomar o exemplo de um agente de monitoramento de latência.

Primeiro, o agente realiza a percepção:

Ele identifica qual serviço está com problema, em qual período e qual o contexto do alerta.

Em seguida, entra na fase de raciocínio:

Com base no estado construído, ele decide consultar métricas ou investigar possíveis causas.

Depois, executa a ação:

Realiza consultas, coleta dados e analisa os resultados.

Na sequência, entra no feedback:

Identifica, por exemplo, que houve degradação após um deploy.

A partir disso, o ciclo recomeça:

Agora o novo contexto é investigar o deploy, e o agente inicia uma nova iteração com informações mais ricas.

Cada iteração melhora a qualidade das decisões. Não porque o modelo mudou, mas porque o estado foi enriquecido.

Engenharia de Software no Centro do Processo

Um dos pontos mais importantes que eu reforço é que o sucesso de um agente não depende do modelo de linguagem utilizado.

O modelo é apenas o mecanismo de raciocínio.

O que realmente determina a qualidade do sistema é a engenharia aplicada ao loop.

Isso inclui:

- Como o estado é construído
- Como as decisões são definidas
- Quais ações são permitidas
- Como o feedback é avaliado
- Quais são os limites de execução

Quando o agente falha, na maioria das vezes, o problema não está na inteligência artificial. Está no desenho do sistema.

Desafio de Modelagem

Para consolidar esse entendimento, eu proponho um exercício prático.

Escolha um agente real, preferencialmente um problema que você já vivenciou no seu dia a dia.

A partir disso, responda às seguintes perguntas:

- O que o agente percebe?
- Qual estado ele constrói?
- O que ele decide?
- Quais ações ele executa?
- Como ele avalia os resultados?
- Quando ele deve parar?

Se você não consegue responder essas perguntas de forma clara, então você ainda não tem um agente definido.

Você tem, no máximo, uma ideia.

Encerramento

Ao final desta aula, eu espero que tenha ficado claro que o Agent Loop é o elemento central na construção de agentes autônomos.

Mais do que entender cada etapa isoladamente, o importante é compreender que estamos projetando um sistema de decisão contínuo.

E, como qualquer sistema crítico, ele precisa ser controlado, observável e previsível.

A partir da próxima aula, eu começo a avançar para a definição de contratos, onde vamos transformar esse modelo conceitual em algo estruturado e implementável.

Capítulo 3: Contratos

Até aqui, eu apresentei o conceito de agentes autônomos e detalhei o funcionamento do Agent Loop. No entanto, ainda falta um passo essencial: transformar esse modelo conceitual em um sistema real.

Saber o que o agente percebe, decide, executa e avalia é apenas o começo. Para que isso funcione de forma consistente, previsível e controlada, é necessário formalizar esse comportamento.

É exatamente nesse ponto que entram os contratos.

O objetivo desta etapa é simples: tirar o agente do campo das ideias e levá-lo para uma estrutura concreta, baseada em especificação.

O Papel da LLM no Sistema

Antes de avançar, é importante esclarecer onde a LLM se encaixa dentro desse modelo.

A LLM é o motor de raciocínio. Ela é responsável por processar contexto e sugerir decisões.

No entanto, ela não define:

- Limites de execução
- Estrutura do sistema
- Ferramentas disponíveis
- Regras de segurança

A LLM opera dentro de um conjunto de regras previamente estabelecidas. Essas regras são os contratos.

Isso muda completamente a forma como pensamos o uso de inteligência artificial. Em vez de confiar no comportamento emergente do modelo, passamos a controlar explicitamente o que ele pode ou não fazer.

Estrutura de um Agente

Um agente, nesse modelo, é representado como um conjunto de arquivos.

Cada arquivo tem uma responsabilidade clara e define um aspecto específico do comportamento do agente.

Esses arquivos não contêm código. Eles contêm especificações.

O runtime do sistema é responsável por interpretar essas especificações e executar o agente com base nelas.

Essa abordagem é conhecida como Spec Driven Agents.

O Arquivo agent.md

O ponto de partida é o arquivo que define a identidade do agente.

Nesse arquivo, eu especifico:

- Nome do agente
- Descrição
- Tipo de operação
- Objetivo
- Contrato de saída

A identidade é fundamental. Sem ela, o agente se torna genérico e perde direcionamento.

O tipo define como o agente opera. Ele pode ser orientado a tarefas, interativo, baseado em objetivos ou totalmente autônomo.

O objetivo, como já discutido anteriormente, é o elemento central. Ele orienta todas as decisões do agente.

O contrato de saída define o que será entregue ao final da execução. Isso inclui formato, estrutura e campos obrigatórios.

Essa definição garante previsibilidade e padronização na saída do sistema.

Contratos de Execução

Além da identidade, o agente possui uma série de contratos que governam seu comportamento durante a execução.

Esses contratos são organizados em uma estrutura específica e cobrem todas as etapas do Agent Loop.

loop.md: Controle do Ciclo

O arquivo loop.md define como o ciclo do agente funciona.

Ele especifica:

- Objetivo da execução
- Número máximo de iterações
- Condições de parada

As condições de parada são particularmente importantes. Entre elas, podemos destacar:

- Objetivo alcançado
- Limite de etapas excedido
- Falta de progresso
- Tempo máximo atingido
- Interrupção por decisão externa

Esse arquivo é responsável por garantir que o agente não entre em execução infinita.

Aqui, começamos a aplicar controle real sobre o comportamento do sistema.

planner.md: Estrutura de Decisão

O planner define como a LLM deve responder em cada iteração do loop.

Ele não é um prompt convencional. Ele é um contrato.

Esse contrato obriga a LLM a produzir saídas estruturadas, incluindo:

- Próxima ação a ser executada
- Ferramenta a ser utilizada
- Argumentos necessários
- Critério de sucesso

Além disso, o planner define regras importantes, como:

- Necessidade de coletar evidências antes de decidir
- Obrigatoriedade de registrar informações antes de encerrar

Isso evita decisões arbitrárias e garante consistência no comportamento do agente.

Skills: O Que o Agente Sabe Fazer

As skills representam as capacidades do agente.

Cada skill define:

- Nome
- Descrição
- Entrada
- Saída

É importante reforçar que a skill não implementa a lógica. Ela define a interface.

Por exemplo, uma skill pode representar a capacidade de consultar métricas, buscar logs ou analisar histórico de deploy.

O agente não executa diretamente essas ações. Ele decide quando utilizá-las.

Toolbox: O Que o Agente Pode Fazer

Existe uma diferença fundamental entre saber e poder.

A skill define o que o agente sabe fazer. A toolbox define o que ele pode fazer.

A toolbox lista explicitamente as ferramentas disponíveis para execução.

Se uma ferramenta está definida na skill, mas não está na toolbox, o agente não pode utilizá-la.

Isso cria uma camada adicional de controle e segurança.

executor.md: Execução Controlada

O executor define como as ações são executadas em tempo de execução.

Ele é responsável por:

- Validar entradas
- Executar ferramentas
- Tratar falhas
- Definir comportamento em caso de erro

Esse contrato garante que a execução siga padrões definidos e evita comportamentos inesperados.

rules.md: Segurança e Limites

O arquivo de regras é um dos mais importantes do sistema.

Ele define:

- Limites de execução
- Restrições por ferramenta
- Ações que exigem confirmação
- Comportamentos obrigatórios

Por exemplo:

- Limite de chamadas de uma ferramenta
- Obrigatoriedade de gerar relatório antes de finalizar
- Necessidade de confirmação humana para ações críticas

Esse arquivo é o principal mecanismo de proteção do agente.

Sem ele, o sistema se torna imprevisível e potencialmente perigoso.

hooks.md: Observabilidade

Os hooks permitem interceptar eventos durante a execução do agente.

Eles podem ser utilizados para:

- Registrar logs
- Emitir alertas
- Monitorar comportamento

Esses pontos de extensão são fundamentais para garantir observabilidade.

Sem observabilidade, não é possível entender o comportamento do agente nem evoluí-lo de forma estruturada.

memory.md: Gestão de Contexto

A memória define o que o agente deve lembrar durante sua execução.

Ela não é um armazenamento permanente. É um contexto de curto prazo.

Normalmente, inclui:

- Resultados de ações
- Decisões tomadas
- Evidências coletadas
- Erros encontrados

Ao mesmo tempo, evita armazenar:

- Prompts completos
- Dados redundantes

A memória deve ser enxuta e relevante.

Sem ela, o agente perde contexto e tende a repetir erros.

Integração com o Runtime

Todos esses contratos são interpretados por um runtime.

O runtime é responsável por:

- Executar o Agent Loop
- Consultar a LLM
- Chamar ferramentas
- Avaliar resultados

O ponto mais importante é que o runtime não contém lógica específica de negócio.

Ele apenas executa o que está definido nos contratos.

Isso permite reutilização e padronização.

Observabilidade e Evolução

Uma das grandes vantagens desse modelo é a rastreabilidade.

Cada decisão, cada ação e cada resultado são registrados.

Isso permite:

- Analisar comportamento
- Identificar falhas
- Ajustar contratos
- Evoluir o agente

O desenvolvimento deixa de ser tentativa e erro e passa a ser um processo iterativo e controlado.

Ciclo de Evolução do Agente

O processo de desenvolvimento segue um ciclo claro:

- Definir contratos
- Executar o agente
- Observar comportamento
- Ajustar especificações

Esse ciclo se repete até que o agente atinja o nível de qualidade desejado.

Diagnóstico de Problemas

Um dos benefícios dessa abordagem é a facilidade de diagnóstico.

Se o agente não decide, o problema tende a estar no planner.

Se não executa, o problema pode estar nas skills ou na toolbox.

Se não para, o problema está nas rules.

Se não há visibilidade, o problema está nos hooks.

Essa separação de responsabilidades facilita a manutenção e evolução do sistema.

Encerramento

Ao final desta etapa, o ponto mais importante é compreender que agentes autônomos não são construídos apenas com código.

Eles são definidos por especificações.

O código executa. A especificação governa.

Essa mudança de paradigma é o que permite construir sistemas mais confiáveis, previsíveis e escaláveis.

Na próxima etapa, o foco passa a ser o runtime, onde eu começo a mostrar como essas especificações são interpretadas e executadas na prática.

Capítulo 4: Runtime

Até este ponto, eu defini um agente completo utilizando apenas contratos. Estructurei comportamento, limites, decisões e capacidades sem escrever uma única linha de código.

No entanto, isso naturalmente levanta uma pergunta essencial: como esses arquivos se transformam em execução real?

É exatamente isso que eu passo a explorar agora. O runtime é o motor responsável por interpretar os contratos e executar o agente.

Ele não contém lógica de negócio específica. Ele não sabe o que é latência, incidente ou deploy. Ele apenas lê especificações e executa conforme definido.

Essa característica é o que torna o sistema genérico e reutilizável.

Estrutura do Runtime

O runtime é composto por um conjunto de módulos escritos em Python.

Cada módulo possui uma responsabilidade específica dentro do ciclo de execução do agente. De forma geral, podemos organizar esses componentes em:

- Carregamento de contratos
- Orquestração do ciclo
- Planejamento (uso da LLM)
- Execução de ações
- Telemetria

Esses módulos trabalham em conjunto para transformar especificações declarativas em comportamento executável.

Carregamento de Contratos

O ponto de partida da execução é o carregamento dos contratos.

O runtime percorre os arquivos definidos para o agente e extrai os blocos estruturados em YAML contidos dentro dos arquivos Markdown.

Essa separação é intencional:

- Markdown representa a documentação legível por humanos
- YAML representa a estrutura interpretável pela máquina

Após a leitura, o runtime constrói um dicionário contendo todos os contratos do agente.

Esse dicionário passa a ser a base de toda a execução.

Construção do Estado Inicial

Com os contratos carregados, o runtime cria o estado inicial do agente.

Esse estado é completamente neutro:

- Nenhuma decisão foi tomada

- Nenhuma ferramenta foi executada
- Nenhum token foi consumido
- O histórico está vazio

Ele representa uma folha em branco estruturada a partir das especificações.

A partir desse ponto, o ciclo do agente pode começar.

Orquestração do Agent Loop

O coração do runtime é o módulo responsável por orquestrar o ciclo do agente.

Esse módulo implementa o Agent Loop, seguindo as etapas:

- Percepção
- Planejamento
- Ação
- Avaliação

A cada iteração, o runtime executa essas fases e, ao final, decide se deve continuar ou encerrar.

Além disso, a cada ciclo, ele valida condições críticas:

- Limite de tempo
- Limite de tokens
- Limite de uso de ferramentas
- Falta de progresso

Essas validações são baseadas diretamente nas regras definidas nos contratos.

Planejamento e Integração com a LLM

O planejador é o componente responsável por interagir com a LLM.

Ele possui duas funções principais:

Construção da Percepção

O runtime monta o contexto que será enviado para a LLM.

Esse contexto inclui:

- Entrada original
- Estado atual
- Histórico de decisões
- Resultados anteriores
- Objetivo do agente

Esse conjunto de informações define a qualidade da decisão.

Se o contexto for pobre ou inconsistente, a decisão também será.

Geração do Plano

Com o contexto montado, o runtime envia uma requisição para a LLM.

A resposta não é um texto livre. Ela deve seguir o contrato definido no planner.

Isso significa que a LLM retorna um JSON estruturado contendo:

- Ação a ser executada
- Ferramenta escolhida
- Argumentos
- Critério de sucesso

Se a LLM não estiver disponível, o runtime utiliza um fallback para simular decisões, permitindo testes sem consumo de recursos externos.

Construção das Ferramentas

Um ponto importante é entender como as ferramentas são executadas.

Nos contratos, eu defini apenas interfaces. Não implementei nenhuma lógica real.

O runtime resolve isso dinamicamente.

Ele constrói funções executáveis com base nas definições das skills.

Essas funções podem:

- Gerar dados simulados via LLM
- Utilizar mocks em caso de fallback

Essa abordagem permite prototipação rápida.

Em ambientes de produção, essas implementações podem ser substituídas por integrações reais, como APIs de monitoramento, logs ou sistemas externos.

Execução Controlada

O executor é responsável por garantir que a execução ocorra de forma segura e consistente.

Ele realiza diversas validações antes e durante a execução:

Validação de Payload

Antes de executar uma ação, o runtime valida:

- Tipos de dados
- Estrutura dos argumentos
- Compatibilidade com a interface definida

Se houver inconsistência, a execução é interrompida.

Execução com Retry

Caso uma ação falhe, o runtime pode tentar executá-la novamente, conforme regras definidas.

Avaliação do Resultado

Após a execução, o resultado é classificado como:

- Completo
- Parcial
- Falho

Essa classificação alimenta o próximo ciclo do agente.

Feedback e Evolução do Estado

O feedback é incorporado ao estado do agente.

Isso significa que cada iteração adiciona informações ao histórico, permitindo decisões mais precisas nas próximas etapas.

O agente não melhora porque o modelo muda, mas porque o contexto evolui.

Essa é uma característica fundamental do sistema.

Circuit Breaker

Um dos mecanismos mais importantes do runtime é o Circuit Breaker.

Ele atua como uma camada de proteção contra falhas inesperadas, como:

- Respostas inválidas da LLM
- Ferramentas inexistentes
- Estruturas malformadas
- Erros de execução

Quando acionado, o Circuit Breaker impede que o sistema execute ações perigosas ou inconsistentes.

Ele também permite que o agente continue operando com respostas controladas.

Esse tipo de mecanismo é essencial para ambientes produtivos.

Telemetria e Observabilidade

A telemetria é responsável por registrar tudo o que acontece durante a execução.

Ela coleta:

- Eventos do ciclo
- Tempo de cada etapa

- Consumo de tokens
- Uso de ferramentas
- Falhas e exceções

Essas informações são armazenadas em um arquivo estruturado, geralmente no formato JSON.

Esse arquivo permite:

- Auditoria completa
- Análise de desempenho
- Diagnóstico de falhas

Diferente de logs simples, aqui temos observabilidade estruturada.

Fluxo Completo de Execução

Quando o agente é executado, o fluxo segue uma sequência bem definida:

Primeiro, os contratos são carregados e transformados em um modelo interno.

Em seguida, o estado inicial é criado.

Depois disso, o loop se inicia:

- O contexto é montado
- A LLM gera um plano
- O plano é validado
- A ação é executada
- O resultado é avaliado

Esse ciclo se repete até que:

- O objetivo seja atingido
- Um limite seja alcançado

Durante todo o processo, a telemetria registra cada passo.

Separação entre Especificação e Execução

Um dos pontos mais importantes desse modelo é a separação entre especificação e execução.

Os contratos definem o comportamento.

O runtime executa esse comportamento.

Essa separação permite:

- Reutilização do motor
- Padronização de agentes
- Facilidade de manutenção
- Evolução controlada

O sistema deixa de ser dependente de implementações específicas e passa a ser orientado por definição.

Encerramento

Ao compreender o runtime, fica claro que o agente não está no código.

O agente está na especificação.

O runtime é apenas o executor.

Essa inversão de responsabilidade é o que permite escalar a construção de agentes de forma consistente.

A partir daqui, o próximo passo natural é aprofundar a observabilidade, analisando como interpretar os dados gerados e como utilizá-los para evoluir os agentes.

Capítulo 5: Observabilidade

Ao longo das aulas anteriores, eu construí um agente completo, defini seu comportamento por contratos e entendi como o runtime executa esse ciclo. No entanto, surge uma questão fundamental:

Como saber se o agente está funcionando corretamente?

Não basta apenas executar o agente. É necessário compreender suas decisões, validar seu comportamento e identificar falhas de forma precisa.

É exatamente nesse ponto que entra a observabilidade.

Limitações dos Logs Tradicionais

Na engenharia de software tradicional, utilizamos logs para entender o comportamento de um sistema.

Um sistema convencional segue um fluxo simples:

- Recebe uma requisição
- Processa
- Retorna uma resposta

Os logs registram esse fluxo e, na maioria dos casos, são suficientes para diagnóstico.

No entanto, agentes autônomos não funcionam dessa forma.

Um agente:

- Recebe um contexto
- Toma decisões
- Executa ações
- Avalia resultados
- Decide novamente

Esse ciclo gera múltiplas decisões intermediárias.

Perguntas fundamentais passam a surgir:

- Quantas decisões o agente tomou?
- Por que escolheu determinada ação?
- Qual foi o critério de sucesso?
- Quanto custou cada etapa?

Logs tradicionais não respondem essas perguntas.

O Conceito de Trace

Para resolver esse problema, utilizamos o conceito de trace.

O trace é o registro completo de uma execução.

Ele representa o caminho completo percorrido pelo agente, incluindo não apenas o que aconteceu, mas também o porquê das decisões.

Diferente de logs simples, o trace fornece:

- Contexto
- Decisão
- Ação
- Resultado
- Avaliação

Essa estrutura permite rastrear cada passo do agente com precisão.

Estrutura do Trace

Cada execução gera um arquivo estruturado, geralmente chamado de trace.json.

Esse arquivo contém informações essenciais, como:

- Identificador único da execução
- Tempo total de execução
- Consumo de tokens
- Lista de etapas

Cada etapa registra quatro elementos fundamentais:

- Percepção: o que o agente sabia naquele momento
- Plano: o que ele decidiu fazer
- Resultado: o que aconteceu após a ação
- Avaliação: se a ação foi bem-sucedida

Essa estrutura transforma o processo de debug.

Em vez de investigar no escuro, é possível identificar exatamente onde e por que o erro ocorreu.

Métricas de Saúde

Além do detalhamento por etapa, o trace fornece métricas de saúde da execução.

Entre elas:

- Taxa de sucesso das ferramentas
- Número de ativações do Circuit Breaker
- Falhas de validação de payload

Essas métricas não apenas indicam que houve um problema, mas apontam diretamente onde ele está.

Por exemplo:

- Baixa taxa de sucesso indica problemas nas ferramentas
- Circuit Breaker ativado indica falha na decisão ou na LLM
- Erros de payload indicam inconsistência nas skills

Essa relação direta entre métrica e causa reduz drasticamente o tempo de diagnóstico.

Análise de Performance

Outro aspecto crítico é a análise de performance.

Saber que uma execução demorou não é suficiente. É necessário entender onde o tempo foi consumido.

O trace registra o tempo de cada fase do ciclo:

- Percepção
- Planejamento
- Ação
- Avaliação

Normalmente, os maiores custos estão nas etapas que envolvem chamadas à LLM.

Isso permite identificar gargalos com precisão e orientar otimizações.

Dashboard em Tempo Real

Além da análise posterior via trace, o runtime oferece um dashboard em tempo real.

Esse dashboard exibe informações como:

- Progresso da execução
- Consumo de tokens
- Uso de ferramentas
- Qualidade das avaliações
- Alertas e falhas

Essa visualização permite acompanhar o comportamento do agente enquanto ele está em execução.

Interpretação dos Indicadores

Cada indicador do dashboard possui um significado claro:

- Progresso indica em qual etapa do ciclo o agente está
- Tokens mostram o consumo em relação ao limite
- Ferramentas indicam quais ações foram ou devem ser executadas
- Qualidade classifica os resultados obtidos
- Alertas indicam falhas críticas

Essa combinação fornece uma visão operacional completa.

Análise Automatizada com Agentes

Uma evolução natural da observabilidade é automatizar a análise.

Para isso, eu introduzo um agente específico: o analisador de trace.

Esse agente recebe como entrada o trace de outro agente e realiza uma avaliação completa.

Ele possui habilidades como:

- Análise de saúde
- Avaliação de performance
- Verificação de conformidade
- Detecção de anomalias
- Geração de recomendações

Esse processo elimina a necessidade de análise manual.

Funcionamento do Agente Analisador

O agente analisador segue o mesmo modelo:

- Recebe o trace como entrada
- Executa o Agent Loop
- Aplica suas skills
- Gera um relatório estruturado

Esse relatório pode incluir:

- Diagnóstico do comportamento
- Identificação de gargalos
- Violações de regras
- Recomendações de melhoria

Diagnóstico e Recomendações

A análise automatizada permite identificar problemas como:

- Ferramentas obrigatórias não executadas
- Gargalos em etapas específicas
- Execuções improdutivas
- Uso ineficiente de recursos

Além disso, o agente fornece recomendações práticas, como:

- Ajustar contratos
- Refinar regras
- Otimizar uso da LLM
- Revisar fluxo de execução

Esse tipo de feedback acelera significativamente a evolução do sistema.

Os Quatro Níveis de Observabilidade

A observabilidade no framework pode ser entendida em quatro níveis:

Hooks

Eventos disparados em tempo real durante a execução.

Dashboard

Visualização em tempo real dos indicadores principais.

Trace

Registro completo da execução para análise detalhada.

Análise Automatizada

Interpretação inteligente dos dados gerados.

Cada nível possui um propósito específico, desde monitoramento imediato até análise profunda.

Importância em Produção

Em ambientes reais, a ausência de observabilidade torna o sistema praticamente impossível de manter.

Sem visibilidade:

- Não é possível entender falhas
- Não há rastreabilidade
- O debug se torna tentativa e erro

Com observabilidade:

- Cada decisão é rastreável
- Cada erro tem causa identificável
- Cada melhoria pode ser orientada por dados

Essa diferença define se um agente é experimental ou pronto para produção.

Encerramento

Ao final desta etapa, fica claro que observabilidade não é um complemento.

Ela é um requisito essencial.

Agentes autônomos tomam decisões. E toda decisão precisa ser compreendida, auditada e validada.

A partir daqui, o desenvolvimento de agentes deixa de ser baseado em tentativa e erro e passa a ser guiado por evidências.

Capítulo 6: Tipos de Agentes e Projeto de Portfólio

Antes de avançar, eu considero importante consolidar o caminho percorrido até aqui.

Na primeira etapa, eu apresentei o conceito de agentes autônomos e a diferença fundamental entre execução e decisão. Em seguida, detalhei o Agent Loop, que estrutura o comportamento do agente em ciclos contínuos de percepção, planejamento, ação e avaliação.

Na sequência, formalizei esse comportamento por meio de contratos, utilizando a abordagem Spec Driven Agents. Depois, mostrei como o runtime interpreta esses contratos e executa o agente. Em seguida, aprofundei a observabilidade, demonstrando como rastrear decisões e diagnosticar falhas.

Agora, eu amplio esse entendimento apresentando os diferentes tipos de agentes e como utilizá-los na prática, especialmente na construção de projetos de portfólio.

A Importância do Tipo de Agente

Até aqui, eu trabalhei com um único tipo de agente: o Task-Based.

No entanto, esse é apenas um dos possíveis modos de operação.

O mesmo runtime, com os mesmos contratos estruturais e o mesmo loop de execução, pode suportar comportamentos completamente diferentes. O que define esse comportamento é um único campo dentro do contrato principal: o tipo do agente.

Esse campo altera a forma como o contexto é interpretado e como as decisões são tomadas.

Eu costumo comparar isso com a troca de marchas em um carro. O motor é o mesmo, mas o comportamento muda completamente dependendo da marcha utilizada.

Os Quatro Tipos de Agentes

Existem quatro tipos principais de agentes que podem ser utilizados.

Task-Based

O agente do tipo Task-Based é o mais simples.

Ele recebe uma tarefa bem definida, executa e retorna um resultado.

Não há decomposição, não há questionamentos e não há iteração complexa. O fluxo é direto.

Esse tipo de agente é ideal para cenários onde:

- A entrada é clara
- O objetivo é específico
- Não há ambiguidade

Um exemplo clássico é o MonitorAgent, que recebe um alerta e retorna um diagnóstico.

Agente Interativo

O agente interativo é utilizado quando a entrada não é suficientemente clara.

Nesse caso, o agente não executa imediatamente. Ele faz perguntas para obter mais contexto.

Esse comportamento é essencial para evitar decisões incorretas baseadas em informações incompletas.

Na prática, ele:

- Identifica lacunas de informação
- Interrompe o fluxo
- Solicita dados adicionais
- Continua a execução após obter respostas

Esse tipo de agente simula o comportamento de um especialista que precisa investigar antes de agir.

Agente Orientado a Objetivo

O agente orientado a objetivo representa um avanço significativo em complexidade.

Ele recebe um objetivo amplo e realiza a decomposição desse objetivo em etapas executáveis.

Esse processo inclui:

- Identificação de subobjetivos
- Planejamento estruturado
- Execução encadeada
- Reavaliação contínua

Um exemplo prático é o Backlog Decomposer, que transforma um requisito em:

- Épicos
- Histórias
- Critérios de aceite
- Riscos
- Perguntas

Esse tipo de agente é ideal para cenários onde:

- O problema é complexo
- O objetivo não está detalhado
- É necessário planejamento estruturado

Agente Autônomo

O agente autônomo opera de forma independente.

Ele não depende de uma solicitação direta de um usuário. Em vez disso, ele responde a eventos.

Esses eventos funcionam como gatilhos.

Quando um evento ocorre, o agente:

- Interpreta o contexto

- Toma decisões
- Executa ações dentro de limites definidos

No entanto, é fundamental entender que autonomia não significa liberdade total.

Ações sensíveis exigem confirmação humana.

Isso garante segurança e evita impactos críticos em produção.

O Que Não Muda

Apesar das diferenças de comportamento, alguns elementos permanecem constantes:

- O runtime é o mesmo
- O Agent Loop é o mesmo
- A estrutura de contratos é a mesma
- A observabilidade é a mesma

Isso demonstra a força da arquitetura: um único motor capaz de executar múltiplos comportamentos.

Comparação entre os Tipos

Cada tipo de agente possui características específicas:

- Task-Based prioriza eficiência
- Interativo prioriza clareza
- Orientado a objetivo prioriza completude
- Autônomo prioriza segurança

A escolha do tipo correto depende do problema que está sendo resolvido.

Desenvolvimento Orientado a Contratos

Um dos pontos mais importantes desta etapa é a mudança na forma de desenvolvimento.

Eu não estou mais ajustando código. Estou ajustando comportamento.

O ciclo de desenvolvimento passa a ser:

- Definir contratos
- Executar o agente
- Observar o trace
- Ajustar a especificação

Esse ciclo é repetido até atingir o comportamento desejado.

Esse processo é conhecido como Contract Driven Development.

Validação de Contratos

Antes de executar o agente, é possível validar os contratos.

Essa validação verifica:

- Consistência entre skills e toolbox
- Existência de ferramentas obrigatórias
- Integridade das definições

Esse processo evita erros em tempo de execução.

É equivalente a um compilador na engenharia tradicional.

Construção de um Agente do Zero

A construção de um agente segue uma sequência estruturada:

Primeiro, eu crio a pasta do agente e os arquivos de contrato.

Em seguida, defino o agent.md, contendo:

- Nome
- Descrição
- Tipo
- Objetivo
- Formato de saída

Depois, configuro os demais contratos:

- loop.md para controle do ciclo
- planner.md para regras de decisão
- executor.md para execução
- toolbox.md para permissões
- skills.md para capacidades
- rules.md para limites
- hooks.md para observabilidade
- memory.md para contexto

Após isso, valido os contratos e executo o agente.

Projeto de Portfólio

A etapa final do módulo é a construção de um projeto real.

O objetivo é aplicar todo o conhecimento adquirido em um cenário prático.

Algumas sugestões incluem:

- Triagem de incidentes
- Revisão de pull requests
- Geração de contratos de API
- Criação de runbooks
- Decomposição de backlog
- Validação de dados
- Análise de compliance
- Onboarding de desenvolvedores

Esses projetos refletem problemas reais do mercado.

O Que Entregar

Um projeto completo deve conter:

- Contratos bem definidos
- Execução via runtime

- Trace de execução
- Análise automatizada
- Versionamento

Não há necessidade de código de negócio.

O foco está no comportamento do agente.

Encerramento do Módulo

Ao final deste módulo, eu construí um entendimento completo sobre agentes autônomos.

Eu passei por:

- Conceito de agentes
- Agent Loop
- Contratos
- Runtime
- Observabilidade
- Tipos de agentes

Mais do que isso, eu estabeleci uma nova forma de pensar software.

Não se trata apenas de executar instruções.

Trata-se de projetar sistemas capazes de decidir.

Essa mudança de paradigma é o que diferencia o uso superficial de inteligência artificial da construção de sistemas realmente inteligentes.

Módulo 02: Raciocínio e Tomada de Decisão em Agentes

Capítulo 1: Arquiteturas Cognitivas

Ao iniciar esta nova unidade, eu parto de um ponto essencial: até aqui, eu construí agentes que funcionam. Eles executam tarefas, seguem contratos, utilizam ferramentas e produzem resultados.

No entanto, surge uma questão mais profunda e determinante:

O agente está pensando da forma correta para o tipo de problema que recebe?

Essa pergunta muda completamente o nível da discussão. Não se trata mais de fazer funcionar, mas de fazer da melhor forma possível.

O Problema Central

Dois agentes podem receber:

- O mesmo contexto
- As mesmas ferramentas
- O mesmo objetivo

E ainda assim produzir resultados completamente diferentes.

A diferença está na forma como o raciocínio é estruturado.

Isso é o que define a arquitetura cognitiva.

O Que é Arquitetura Cognitiva

Arquitetura cognitiva é a forma como o agente organiza seu raciocínio antes de agir.

Não se trata do que ele faz, mas de como ele pensa sobre o que deve fazer.

Essa distinção é fundamental.

Dois agentes podem executar as mesmas ações, mas com processos mentais completamente diferentes.

Analogia com Engenharia de Software

Eu gosto de pensar nisso como dois perfis distintos de resolução de problemas:

Um desenvolvedor começa executando imediatamente, explorando o problema passo a passo.

Outro desenvolvedor analisa o cenário completo antes de executar qualquer ação.

Ambos podem chegar ao resultado, mas:

- Um consome mais recursos
- Outro é mais previsível
- Um explora
- Outro planeja

Essa diferença define eficiência, custo e qualidade.

O Padrão Utilizado Até Aqui

Na unidade anterior, eu utilizei implicitamente um padrão específico de raciocínio.

Esse padrão é conhecido como **ReAct**.

Ele segue um ciclo simples:

- Pensar
- Agir
- Observar
- Repetir

O agente não possui um plano completo desde o início.

Ele descobre o caminho conforme avança.

Limitações do Modelo ReAct

Apesar de poderoso, o ReAct possui limitações claras.

Ele não é adequado quando:

- O processo já é conhecido
- O problema exige planejamento estruturado
- A qualidade da resposta é crítica

Nesses cenários, ele pode:

- Consumir mais tokens
- Executar passos desnecessários
- Produzir resultados inconsistentes

Os Três Padrões Cognitivos

Nesta unidade, eu passo a trabalhar com três arquiteturas principais.

ReAct

O ReAct é baseado em exploração incremental.

O agente:

- Observa o estado atual
- Decide a próxima ação
- Executa
- Reavalia

Ele é ideal quando:

- O caminho é incerto
- A solução depende de descoberta
- Cada passo influencia o próximo

O custo é a falta de previsibilidade.

Plan and Execute

No modelo Plan and Execute, o agente muda completamente sua abordagem.

Ele primeiro constrói um plano completo.

Depois executa esse plano passo a passo.

Esse modelo é ideal quando:

- O processo é conhecido
- As etapas são previsíveis
- É necessário controle e auditabilidade

A principal vantagem é a previsibilidade.

Reflection

A arquitetura Reflection adiciona um novo nível de sofisticação.

O agente:

- Executa
- Avalia o próprio resultado
- Critica sua resposta
- Refina antes de finalizar

Esse modelo é ideal quando:

- A qualidade é crítica
- O erro tem alto custo
- É necessário validar múltiplos cenários

O custo é a latência adicional.

Comparação Entre Arquiteturas

Cada arquitetura resolve um tipo de problema.

ReAct é adequado para exploração.

Plan and Execute é adequado para processos estruturados.

Reflection é adequado para cenários onde a qualidade é prioritária.

A escolha da arquitetura impacta diretamente:

- Custo
- Performance
- Qualidade
- Previsibilidade

Escolha da Arquitetura

A escolha não deve ser baseada em preferência.

Ela deve ser baseada no problema.

Se o processo é conhecido, não faz sentido explorar.

Se o problema é incerto, não faz sentido planejar tudo antes.

Se o custo de erro é alto, não faz sentido não revisar.

Raciocínio Implícito vs Explícito

Até este ponto, o raciocínio do agente estava implícito.

Eu via:

- Ação
- Resultado

Mas não via:

- Por que a ação foi escolhida
- O que o agente sabia
- O que estava faltando

Isso limita completamente o debug.

Introdução ao Reasoning Trace

A solução é tornar o raciocínio explícito.

Para isso, eu adiciono um novo campo no Planner:

- Raciocínio

Esse campo registra:

- O que o agente já sabe
- O que ainda falta
- Por que escolheu aquela ação

Isso transforma completamente a observabilidade.

Estrutura do Novo Planner

O Planner passa a retornar:

- Raciocínio
- Próxima ação
- Ferramenta
- Argumentos
- Critério de sucesso

O raciocínio deixa de ser implícito e passa a ser obrigatório.

Impacto no Runtime

Para suportar essa mudança, o runtime precisa evoluir.

As principais mudanças incluem:

- Leitura dinâmica do formato de saída
- Suporte a múltiplas arquiteturas
- Propagação da arquitetura no estado
- Inclusão do raciocínio no trace

Isso torna o sistema extensível.

Open/Closed Principle

O runtime passa a seguir o princípio Open/Closed.

Ele é:

- Aberto para extensão
- Fechado para modificação

Novas arquiteturas podem ser adicionadas sem alterar o código base.

Execução com Raciocínio

Ao executar o agente com a arquitetura ReAct explícita, eu passo a observar algo novo.

Antes de cada ação, o agente explica seu raciocínio.

Isso inclui:

- Análise do estado atual
- Identificação de lacunas
- Justificativa da decisão

Benefícios do Reasoning Trace

O reasoning trace traz benefícios imediatos.

Eu consigo:

- Identificar erros de raciocínio
- Detectar escolhas incorretas de ferramentas
- Entender falhas de contexto
- Melhorar regras de decisão

Isso reduz drasticamente o tempo de diagnóstico.

Debug Baseado em Raciocínio

Sem reasoning trace:

- Eu vejo o erro

- Não sei a causa

Com reasoning trace:

- Eu vejo o erro
- Entendo exatamente por que ele ocorreu

Isso transforma o processo de debug.

Riscos do ReAct

O ReAct possui um risco importante.

Ele pode entrar em loops exploratórios.

Exemplo:

- Consulta métricas
- Não encontra resposta
- Consulta novamente
- Repete indefinidamente

Isso gera:

- Alto consumo de tokens
- Nenhum progresso

Mecanismos de Controle

Para evitar esse comportamento, eu utilizo:

- Limites de execução
- Detecção de ausência de progresso
- Circuit Breaker

Com o reasoning trace, eu consigo detectar esses loops antes mesmo do sistema interromper.

Evolução da Observabilidade

Antes, o trace mostrava apenas ações.

Agora, ele mostra:

- Pensamentos
- Decisões
- Justificativas

Ele deixa de ser um log e passa a ser um diário de execução.

Impacto na Qualidade

Quando o agente é obrigado a explicar seu raciocínio, a qualidade das decisões melhora.

Isso acontece porque:

- O raciocínio precisa ser coerente
- As decisões precisam ser justificáveis
- O modelo evita escolhas superficiais

Encerramento da Unidade

Ao final desta etapa, eu não apenas construo agentes.

Eu passo a projetar como eles pensam.

Essa é a diferença fundamental.

Eu deixo de ajustar prompts e passo a definir arquiteturas cognitivas.

Isso eleva o nível do desenvolvimento de agentes para um patamar de engenharia.

Capítulo 2: Plan Execute e Reflection

Na etapa anterior, eu implementei a arquitetura ReAct e tornei o raciocínio do agente explícito. Com isso, passei a enxergar como o agente pensa em cada decisão.

Agora, eu avanço para duas novas arquiteturas que resolvem limitações importantes do modelo anterior:

- Plan and Execute
- Reflection

Ambas partem de uma premissa diferente: nem todo problema deve ser resolvido passo a passo.

Limitação do ReAct

O modelo ReAct funciona muito bem quando o problema é incerto.

No entanto, ele possui um custo elevado, pois:

- A LLM é chamada a cada etapa
- O agente decide continuamente
- O caminho não é previsível

Isso gera:

- Alto consumo de tokens
- Maior latência
- Menor previsibilidade

A pergunta que guia esta evolução é simples:

E se o agente já soubesse o caminho completo antes de começar?

Arquitetura Plan and Execute

A arquitetura Plan and Execute muda completamente o fluxo.

Em vez de decidir a cada etapa, o agente:

1. Gera um plano completo
2. Executa esse plano passo a passo
3. Finaliza sem replanejar

Diferença Fundamental

No ReAct:

- A LLM é chamada várias vezes

No Plan and Execute:

- A LLM é chamada apenas uma vez

Essa diferença é crítica.

Eu deixo de ter múltiplas decisões distribuídas e passo a ter uma única decisão estruturada.

Estrutura do Plano

Na primeira chamada, o agente gera um plano completo contendo:

- Lista ordenada de passos
- Objetivo de cada passo
- Ferramenta a ser utilizada
- Argumentos necessários
- Critério de sucesso

Após isso, o runtime assume o controle.

Execução Determinística

Com o plano definido, o agente não precisa mais raciocinar a cada etapa.

Ele apenas executa.

Isso transforma o comportamento do sistema em algo determinístico.

Benefícios do Plan and Execute

Essa arquitetura traz três benefícios principais.

Previsibilidade

Eu sei exatamente o que o agente vai fazer antes da execução.

Eficiência

Menos chamadas à LLM resultam em:

- Menor custo
- Menor latência

Auditabilidade

O plano completo aparece antes da execução.

Isso permite revisão antecipada.

Impacto Prático

Em termos práticos:

- ReAct: múltiplas chamadas
- Plan and Execute: uma única chamada

Isso pode reduzir drasticamente o custo operacional.

Implementação via Contratos

A implementação segue o mesmo padrão utilizado anteriormente.

Eu crio uma nova arquitetura contendo:

- Planner
- Executor

O ponto central está no Planner.

Campos Essenciais

Dois campos são responsáveis pela mudança de comportamento:

Modo de Execução

Define que o runtime não deve chamar a LLM novamente.

Plano Completo

Contém todos os passos ordenados.

Esses dois elementos alteram completamente a execução.

Alterações no Runtime

O runtime passa a:

- Armazenar o plano completo
- Executar cada passo sequencialmente
- Evitar novas chamadas à LLM

A partir da segunda etapa, o consumo de tokens é zero.

Limitação do Plan and Execute

Apesar das vantagens, essa arquitetura possui uma limitação crítica.

Ela não se adapta.

Se o cenário muda durante a execução, o agente não reage.

Problema de Rigidez

Se um passo retorna um resultado inesperado:

- O agente continua executando
- O erro se propaga
- O resultado final pode ser inválido

Isso ocorre porque o plano foi definido previamente.

Exemplo Prático

Imagine que o plano assume a existência de um deploy recente.

Se esse deploy não existir:

- O agente não sabe reagir
- Ele continua executando
- O resultado perde consistência

Lição Importante

O Plan and Execute só funciona bem quando:

- O processo é estável
- As etapas são previsíveis
- O ambiente não muda

Caso contrário, ele falha silenciosamente.

Trade-off

Aqui existe um trade-off claro:

- Eficiência versus adaptação

Quanto mais previsível o processo, melhor o desempenho.

Quanto mais incerto, maior o risco.

Introdução à Arquitetura Reflection

Até agora, tanto ReAct quanto Plan and Execute possuem uma limitação comum.

Nenhum deles revisa o próprio resultado.

O agente executa e finaliza.

Sem validação.

O Problema da Falta de Revisão

Esse comportamento é equivalente a:

- Escrever código
- Não revisar
- Enviar direto para produção

Isso compromete a qualidade.

O Conceito de Reflection

A arquitetura Reflection adiciona uma nova etapa:

Autoavaliação.

Antes de finalizar, o agente analisa sua própria resposta.

Fluxo de Execução

O fluxo passa a ser:

- Executar
- Avaliar
- Criticar
- Corrigir
- Reavaliar

Esse ciclo pode se repetir até atingir qualidade suficiente.

Contrato de Critique

Para suportar esse comportamento, eu introduzo um novo contrato:

- Critique

Esse contrato define como a avaliação deve ocorrer.

Critérios de Avaliação

A avaliação é baseada em três critérios principais.

Corretude

O diagnóstico está correto?

Compleitude

Todas as evidências foram consideradas?

Qualidade

Os dados são específicos e relevantes?

Sistema de Pontuação

Cada execução recebe uma nota.

Se a nota estiver acima do limite:

- O agente finaliza

Caso contrário:

- O agente corrige
- Executa novamente

Limite de Reflexões

Para evitar loops infinitos, existe um limite.

O agente pode tentar corrigir apenas um número definido de vezes.

Isso garante controle.

Funcionamento no Runtime

O runtime intercepta a finalização.

Antes de encerrar:

- Executa o Critique
- Avalia o resultado
- Decide se aceita ou rejeita

Se rejeitar:

- Injeta feedback
- Reexecuta o agente

Benefícios do Reflection

Essa arquitetura traz ganhos claros.

Qualidade

As respostas são mais refinadas.

Consistência

Erros são corrigidos antes da entrega.

Confiabilidade

O agente valida suas próprias decisões.

Custo do Reflection

Esse ganho não é gratuito.

Há aumento em:

- Tokens
- Latência
- Complexidade

Trade-off de Qualidade

O Reflection representa outro trade-off:

- Qualidade versus custo

Ele é ideal quando o erro é caro.

Comparação das Arquiteturas

Neste ponto, eu passo a ter três arquiteturas distintas.

ReAct

- Alta adaptação
- Alto custo
- Baixa previsibilidade

Plan and Execute

- Alta previsibilidade
- Baixo custo
- Baixa adaptação

Reflection

- Alta qualidade
- Custo elevado
- Controle de erro

Escolha da Arquitetura

A escolha depende do problema.

Problemas exploratórios → ReAct

Processos estruturados → Plan and Execute

Alta criticidade → Reflection

Engenharia Baseada em Dados

A decisão não deve ser baseada em opinião.

Deve ser baseada em:

- Métricas
- Custo
- Latência
- Qualidade

Essa abordagem transforma o desenvolvimento em engenharia real.

Encerramento

Ao final desta etapa, eu deixo de trabalhar com um único modelo de raciocínio.

Eu passo a escolher conscientemente como o agente pensa.

Essa capacidade é o que diferencia:

- Uso básico de IA
- Engenharia de sistemas inteligentes

Agora, o agente não apenas executa.

Ele:

- Planeja
- Decide
- Avalia
- Corrige

E isso define um novo nível de maturidade na construção de software.

Capítulo 3: Evals e Frameworks de Mercado

Neste ponto do módulo, eu já tenho três arquiteturas em mãos:

- ReAct
- Plan and Execute

- Reflection

Naturalmente, surge a tendência de escolher uma delas com base em percepção.

No entanto, eu preciso ser direto: opinião não é evidência.

Se eu testei apenas um cenário, eu não tenho base para decidir. Eu tenho apenas um palpite.

E engenharia de software não é construída sobre palpites.

A Necessidade de Avaliação

Para sair do achismo, eu introduzo um conceito fundamental: eval.

Eval é uma avaliação automatizada do comportamento do agente.

O objetivo é simples:

Substituir opinião por dados.

O Que é um Eval

Um eval é composto por três elementos principais:

Dataset

Um conjunto de cenários que representam diferentes situações.

Métricas

Indicadores que permitem medir o desempenho do agente.

Limites

CrITÉrios que determinam se o agente está aprovado ou reprovado.

Esse conjunto define uma prova estruturada.

Diferença para Testes Tradicionais

É importante destacar que eval não é teste unitário.

Testes unitários validam código.

Evals validam comportamento.

Aqui, eu não estou testando uma função. Estou testando como o agente se comporta em diferentes cenários.

Construção do Dataset

O primeiro passo é construir um dataset.

Esse dataset contém múltiplos cenários, por exemplo:

- Cenários simples
- Cenários intermediários
- Cenários complexos

Cada cenário define:

- Entrada
- Dificuldade
- Ferramentas esperadas

Isso garante cobertura.

O Papel do Dataset

O dataset funciona como uma prova.

Cada arquitetura é um aluno.

Todos resolvem a mesma prova.

E o resultado mostra quem performa melhor.

Definição de Métricas

O segundo passo é definir métricas.

Essas métricas permitem avaliar o desempenho do agente de forma objetiva.

Entre as principais métricas, temos:

- Taxa de conclusão
- Número médio de etapas
- Consumo de tokens
- Tempo de execução
- Taxa de sucesso das ferramentas
- Cobertura de ferramentas
- Uso de Circuit Breaker
- Número de reflexões

Essas métricas representam diferentes dimensões do comportamento.

Definição de Limites

O terceiro passo é definir limites mínimos.

Por exemplo:

- Pelo menos 80% dos cenários devem ser concluídos
- Pelo menos 90% das ferramentas devem funcionar corretamente
- Pelo menos 75% das ferramentas esperadas devem ser utilizadas

Esses limites transformam métricas em critérios de aprovação.

Execução do Benchmark

Com dataset, métricas e limites definidos, eu executo o benchmark.

O benchmark roda o agente em todos os cenários.

Para cada execução, ele coleta:

- Dados de performance
- Dados de comportamento

- Dados de consumo

Ao final, ele gera um relatório.

Estrutura do Benchmark

O benchmark realiza:

- Execução de cada cenário
- Coleta de métricas
- Comparação com limites
- Registro de violações

Esse processo é repetido para cada arquitetura.

Relatório Comparativo

O resultado final é um relatório comparativo.

Esse relatório mostra:

- Desempenho de cada arquitetura
- Melhores valores por métrica
- Violações de limites
- Veredito final

Agora a decisão deixa de ser subjetiva.

Interpretação dos Resultados

Ao analisar os resultados, padrões claros emergem.

Plan and Execute

- Menor consumo de tokens
- Maior velocidade
- Menor adaptação

ReAct

- Maior cobertura de ferramentas
- Melhor adaptação
- Maior custo

Reflection

- Melhor qualidade de saída
- Maior custo
- Maior latência

O Verdadeiro Resultado

A pergunta “qual é o melhor?” deixa de existir.

A resposta correta passa a ser:

Depende do problema.

Mas agora esse “depende” é baseado em dados.

Trade-offs Reais

Cada arquitetura resolve um tipo de necessidade:

- Eficiência → Plan and Execute
- Flexibilidade → ReAct
- Qualidade → Reflection

A escolha passa a ser estratégica.

Diagnóstico de Falhas

Quando uma arquitetura falha, isso não é erro.

É informação.

Por exemplo:

Se o Plan and Execute não utiliza uma ferramenta esperada, isso confirma sua rigidez.

Se o ReAct consome muitos tokens, isso confirma seu comportamento exploratório.

Os dados validam a teoria.

Método Científico Aplicado

O processo completo segue um método claro:

1. Definir hipótese
2. Criar dataset
3. Definir métricas
4. Definir limites
5. Executar benchmark
6. Comparar resultados
7. Tomar decisão baseada em evidência

Esse é o método científico aplicado à engenharia de agentes.

Da Teoria ao Mercado

Tudo que foi construído até aqui existe em frameworks reais.

O que muda não são os conceitos, mas a forma de implementação.

Equivalência com Frameworks

Os conceitos que eu utilizei possuem equivalentes diretos:

- Skills → Tools
- Planner → Prompt Template / Agent
- Executor → Agent Executor
- Loop → Runtime / Graph

- Rules → Configurações / Guardrails
- Observabilidade → Callbacks / Tracing
- Eval → Ferramentas de avaliação

Os nomes mudam, mas a estrutura permanece.

LangChain e LangGraph

Nos frameworks modernos:

- LangChain utiliza composição de classes
- LangGraph utiliza grafos de estado

Ambos implementam os mesmos conceitos fundamentais.

Importância dos Fundamentos

Frameworks mudam constantemente.

Conceitos permanecem.

Quem entende apenas o framework se torna dependente.

Quem entende o conceito se torna independente.

Benefício Prático

Ao dominar os fundamentos:

- Migrações se tornam rápidas
- Novos frameworks são compreendidos facilmente
- Decisões são mais precisas

A curva de aprendizado deixa de ser um problema.

Ferramentas de Mercado para Evals

Além da implementação manual, existem ferramentas específicas para evals.

Um exemplo é:

- Promptfoo

Essa ferramenta permite:

- Comparar modelos
- Testar prompts
- Avaliar cenários em escala

Ela funciona de forma semelhante ao benchmark implementado.

Evolução do Engenheiro

Neste ponto, ocorre uma mudança fundamental.

Eu deixo de:

- Testar manualmente
- Escolher por intuição

E passo a:

- Medir comportamento
- Tomar decisões baseadas em dados

Isso caracteriza engenharia de verdade.

Encerramento da Unidade

Ao final desta unidade, eu construí:

- Três arquiteturas cognitivas
- Um sistema de avaliação estruturado
- Um método científico de decisão
- Um mapeamento para frameworks de mercado

Mais do que isso, eu desenvolvi uma nova forma de pensar.

Eu não escolho mais baseado em opinião.

Eu escolho baseado em evidência.

E essa é a diferença entre:

- Usar inteligência artificial
- Construir sistemas inteligentes

Módulo 03: Integração com o Mundo Real e Ferramentas

Capítulo 1: De Mock para Real

Até este ponto, eu construí agentes completos, com contratos bem definidos, runtime funcional, observabilidade estruturada e múltiplas arquiteturas cognitivas.

Esses agentes funcionam. Eles executam tarefas, tomam decisões e produzem resultados.

No entanto, existe um problema crítico que precisa ser enfrentado:

Todos os dados utilizados até agora são fictícios.

O Problema dos Dados Mockados

Durante as unidades anteriores, eu utilizei mocks para simular:

- Métricas
- Logs
- Histórico de deploys

Esses dados foram gerados pela LLM.

Isso é extremamente útil para aprendizado e validação de fluxo. Porém, em produção, esse modelo se torna inviável.

Um agente que toma decisões com base em dados inventados não é confiável.

Riscos do Uso de Mock em Produção

Quando um agente opera com dados fictícios, dois cenários perigosos podem ocorrer:

Falso Positivo

O agente detecta um problema que não existe.

Por exemplo, indicar alta latência quando o sistema está saudável.

Falso Negativo

O agente ignora um problema real.

Por exemplo, não detectar uma falha crítica em produção.

Ambos os casos geram impacto direto:

- Decisões incorretas
- Perda de confiança
- Risco operacional

A Limitação do Protótipo

Um agente baseado em mock é apenas um protótipo.

Ele demonstra fluxo, mas não entrega valor real.

Se o agente não acessa dados reais, ele não pode tomar decisões reais.

E se não pode tomar decisões reais, ele não é um sistema de produção.

A Solução: Padrão Adapter

Para resolver esse problema, eu introduzo o padrão Adapter.

O objetivo é conectar o agente a fontes reais de dados, sem alterar sua arquitetura.

Princípio Fundamental

O mais importante aqui é:

Eu não altero o contrato.
Eu altero a implementação.

Isso preserva toda a estrutura construída anteriormente.

Evolução do Fluxo

Antes:

- Skills → LLM → Dados mockados

Agora:

- Skills → Adapter → APIs reais / Banco de dados / Serviços externos

Essa mudança ocorre apenas na camada de execução.

Tipos de Implementação

Para suportar essa evolução, eu introduzo um novo campo nos contratos: tipo de implementação.

Esse campo define como a skill será executada.

Existem quatro tipos:

Mock

Comportamento original, baseado em LLM.

REST

Chamada a APIs HTTP.

Database

Execução de queries em banco de dados.

MCP

Integração com servidores MCP.

Estrutura Estendida das Skills

As skills passam a conter novos blocos:

- Tipo de implementação
- Configuração de conexão
- Limites de execução

O restante permanece idêntico:

- Nome
- Descrição
- Entrada
- Saída

Isso garante compatibilidade total com o sistema existente.

Desacoplamento de Camadas

Essa abordagem reforça um princípio essencial:

Separação de responsabilidades.

- O Planner decide
- O Executor executa
- O Adapter conecta

Cada camada possui uma função clara.

Funcionamento do Adapter

O Adapter é responsável por:

- Interpretar o contrato
- Construir a requisição
- Executar a chamada externa
- Adaptar a resposta para o formato esperado

Ele não toma decisões. Ele apenas conecta.

Implementação do REST Adapter

O primeiro tipo implementado é o REST Adapter.

Ele permite que o agente consuma APIs reais.

O fluxo é:

1. Ler configuração da skill
2. Montar requisição HTTP
3. Executar chamada
4. Processar resposta
5. Retornar resultado estruturado

API Local para Testes

Para simular integração real, eu utilizo uma API local.

Essa API fornece:

- Métricas
- Logs
- Histórico de deploys

Os dados são fixos e consistentes.

Importância de Dados Consistentes

Diferente do mock, que gera valores aleatórios, a API retorna sempre os mesmos dados.

Isso permite:

- Reprodutibilidade
- Comparação entre execuções
- Validação de comportamento

Integração com o Runtime

O runtime é adaptado para suportar múltiplos tipos de implementação.

Ele utiliza um componente central:

Resolver Adapter

Esse componente:

- Identifica o tipo de implementação
- Direciona para o adapter correto
- Aplica fallback em caso de erro

Fallback e Resiliência

Se o adapter falhar:

- O sistema pode retornar para mock
- O agente continua executando

Esse comportamento garante robustez.

Controle de Limites

As chamadas externas introduzem novos desafios:

- Rate limiting
- Timeout
- Falhas de rede

Para isso, os contratos incluem limites:

- Número de tentativas
- Tempo máximo de resposta
- Volume de dados

Esses limites são aplicados antes da execução.

Retry e Timeout

Dois mecanismos são fundamentais:

Retry

Permite repetir chamadas em caso de falha temporária.

Timeout

Evita bloqueios indefinidos.

Esses mecanismos garantem estabilidade.

Observabilidade com Dados Reais

Com dados reais, a observabilidade se torna ainda mais importante.

O trace passa a indicar:

- Origem dos dados
- Tipo de adapter
- Latência da chamada
- Resultado da execução

Isso permite auditoria completa.

Diferença no Trace

Antes:

- Dados variáveis

- Origem desconhecida
- Comportamento inconsistente

Agora:

- Dados consistentes
- Origem rastreável
- Execução previsível

Segurança

Um ponto crítico é o gerenciamento de credenciais.

Regras fundamentais:

- Nunca armazenar secrets no código
- Nunca incluir credenciais em contratos
- Utilizar variáveis de ambiente

Isso protege o sistema.

Integração com Observabilidade Avançada

Para ambientes reais, ferramentas especializadas são recomendadas.

Um exemplo é:

- Langfuse

Essa ferramenta permite:

- Monitorar execuções
- Analisar custo
- Rastrear decisões

Benefícios da Transição

Ao migrar de mock para real, o agente:

- Ganha confiabilidade

- Produz resultados reais
- Pode operar em produção

Essa é a transição mais importante do módulo.

O Que Não Muda

Apesar da mudança, muitos elementos permanecem iguais:

- Contratos
- Planner
- Executor
- Agent Loop
- Arquiteturas cognitivas

Apenas a fonte de dados muda.

Engenharia em Camadas

Essa evolução demonstra um princípio clássico:

Mudança de infraestrutura sem alterar comportamento.

Isso é possível graças ao desacoplamento.

Encerramento da Aula

Ao final desta etapa, eu deixo de trabalhar com simulações.

Eu passo a trabalhar com realidade.

O agente deixa de ser um protótipo e começa a se tornar um sistema de produção.

Essa mudança não é apenas técnica.

Ela representa a diferença entre:

- Demonstrar um conceito
- Entregar valor real

Capítulo 2: Database e MCP

Na etapa anterior, eu realizei a transição do mock para dados reais utilizando o REST Adapter. Com isso, o agente passou a consumir APIs externas e operar com informações consistentes.

Agora, eu avanço para dois cenários ainda mais críticos no contexto de sistemas reais:

- Integração com banco de dados
- Integração padronizada via MCP

Esses dois elementos elevam o nível do agente de forma significativa, principalmente em ambientes corporativos.

O Problema da Integração Direta com Banco

Quando um agente passa a acessar banco de dados, o nível de risco aumenta.

Diferente de APIs, que já possuem camadas de proteção, o banco de dados expõe diretamente:

- Informações sensíveis
- Estruturas internas
- Dados críticos

Se não houver controle adequado, o agente pode:

- Executar consultas incorretas
- Acessar dados indevidos
- Gerar impactos em produção

O Papel do Database Adapter

Para resolver esse problema, eu introduzo o Database Adapter.

Ele atua como uma camada intermediária entre o agente e o banco de dados.

Seu objetivo é:

- Controlar o acesso
- Padronizar consultas
- Garantir segurança

Execução de Queries Parametrizadas

Um dos princípios fundamentais aqui é o uso de queries parametrizadas.

Isso significa que:

- A estrutura da consulta é fixa
- Apenas os parâmetros variam

Essa abordagem evita um problema clássico:

Injeção de SQL.

Proteção Contra SQL Injection

Quando um agente constrói queries dinamicamente, existe o risco de inserção maliciosa.

Por exemplo:

- Alteração de comandos
- Acesso indevido a dados
- Execução de operações destrutivas

Com queries parametrizadas, esse risco é mitigado.

O banco interpreta os parâmetros como dados, não como comandos.

Restrição de Operações

Outro ponto essencial é limitar o tipo de operação.

O Database Adapter deve operar, preferencialmente, em modo:

- Read-only

Isso garante que o agente:

- Consulta dados
- Não altera dados

Essa restrição é fundamental em ambientes produtivos.

Configuração no Contrato

Assim como no REST Adapter, o Database Adapter é definido via contrato.

A skill passa a incluir:

- Tipo de implementação: database
- Query definida
- Parâmetros permitidos
- Limites de execução

Essa definição mantém o padrão declarativo.

Limites e Controle

O acesso ao banco deve respeitar limites claros:

- Tempo máximo de execução
- Número de registros retornados
- Frequência de chamadas

Esses limites evitam:

- Sobrecarga no banco
- Consultas pesadas
- Impacto em performance

Separação de Responsabilidades

Mesmo com acesso ao banco, a arquitetura permanece consistente:

- O Planner decide o que consultar
- O Adapter executa a consulta
- O Executor avalia o resultado

O agente continua sem acesso direto ao banco.

Introdução ao MCP

Após tratar banco de dados, eu avanço para um conceito mais abrangente: MCP.

MCP significa Model Context Protocol.

Ele define um padrão de comunicação entre agentes e ferramentas.

O Problema da Integração Heterogênea

Sem um padrão, cada integração exige:

- Código específico
- Adaptação manual
- Manutenção constante

Isso gera:

- Complexidade
- Acoplamento
- Dificuldade de evolução

O Papel do MCP

O MCP resolve esse problema criando um protocolo padronizado.

Com ele, o agente não precisa conhecer detalhes da implementação.

Ele interage com ferramentas de forma uniforme.

Funcionamento do MCP Adapter

O MCP Adapter atua como uma ponte entre o agente e servidores MCP.

Ele:

- Envia requisições padronizadas
- Recebe respostas estruturadas
- Traduz para o formato do agente

Vantagens do MCP

O uso do MCP traz benefícios importantes:

Padronização

Todas as integrações seguem o mesmo formato.

Reutilização

Ferramentas podem ser compartilhadas entre agentes.

Escalabilidade

Novas integrações são adicionadas sem alterar o agente.

Comparação com REST

Embora REST seja amplamente utilizado, ele possui limitações:

- Cada API possui formato próprio
- Necessidade de adaptação específica

O MCP resolve isso com um contrato comum.

Integração com o Runtime

Assim como os outros adapters, o MCP é integrado ao runtime via:

- Tipo de implementação
- Resolver Adapter

O runtime identifica o tipo e direciona a execução.

Continuidade do Modelo

Mesmo com múltiplos adapters, o modelo permanece consistente:

- Contratos governam comportamento
- Runtime executa
- Adapters conectam

Essa consistência é o que garante escalabilidade.

Segurança em Integrações

Ao trabalhar com banco e MCP, a segurança se torna ainda mais crítica.

Boas práticas incluem:

- Uso de variáveis de ambiente para credenciais
- Restrição de permissões
- Monitoramento de acessos
- Validação de entradas

Esses cuidados evitam vulnerabilidades.

Observabilidade em Integrações

Com múltiplas fontes de dados, o trace se torna ainda mais relevante.

Ele permite identificar:

- Qual adapter foi utilizado
- Tempo de execução
- Sucesso ou falha
- Origem dos dados

Isso facilita o diagnóstico.

Complexidade Controlada

Apesar do aumento de complexidade, a arquitetura continua controlada.

Isso ocorre porque:

- Cada componente possui responsabilidade clara
- As integrações são desacopladas
- O comportamento é definido por contratos

Evolução do Agente

Neste ponto, o agente evolui significativamente.

Ele passa a:

- Consumir APIs
- Consultar banco de dados
- Integrar com sistemas padronizados

Isso o aproxima de um sistema real de produção.

Encerramento

Ao final desta etapa, eu deixo de trabalhar apenas com integrações simples.

Eu passo a lidar com cenários reais e complexos, onde:

- Segurança é essencial
- Padronização é necessária
- Escalabilidade é obrigatória

O agente não apenas executa e decide.

Ele agora se conecta de forma segura e estruturada com o ecossistema ao seu redor.

Capítulo 3: Tool Selection Eval

Ao longo do módulo, eu construí agentes capazes de:

- Executar tarefas
- Utilizar ferramentas
- Integrar com sistemas reais

No entanto, neste ponto surge uma distinção fundamental:

Um agente pode funcionar perfeitamente... e ainda assim estar errado.

Ele pode:

- Executar todas as etapas
- Finalizar com sucesso
- Gerar um relatório coerente

E mesmo assim ter tomado decisões ruins.

O Problema da Escolha de Ferramentas

O ponto crítico está na escolha das ferramentas.

Dado um contexto, o agente precisa decidir:

- Qual ferramenta utilizar
- Em que momento utilizar
- Com quais argumentos

Essa decisão define a qualidade do comportamento.

Exemplo Prático

Considere o seguinte cenário:

Um alerta indica erro 500 intermitente após um deploy.

A entrada menciona explicitamente um deploy.

O comportamento esperado seria:

Consultar histórico de deploy primeiro.

No entanto, um agente pode decidir:

Consultar métricas primeiro.

Ambos funcionam.

Mas apenas um está correto.

O Conceito de Tool Selection Eval

Para medir essa qualidade, eu introduzo o Tool Selection Eval.

Diferente dos benchmarks anteriores, que avaliam o agente completo, aqui eu avalio decisões individuais.

Cada etapa do loop é analisada isoladamente.

O Que Está Sendo Medido

O Tool Selection Eval responde quatro perguntas fundamentais:

Seleção da Ferramenta

O agente escolheu a ferramenta correta?

Precisão dos Argumentos

Os parâmetros utilizados estavam corretos?

Chamadas Desnecessárias

O agente chamou ferramentas que não deveria?

Escolhas Erradas

O agente utilizou ferramentas incorretas?

Essas quatro dimensões definem a inteligência prática do agente.

Métricas Definidas

A partir dessas perguntas, eu estabeleço quatro métricas principais:

- Tool Selection Accuracy
- Argument Accuracy
- Unnecessary Calls Rate
- Wrong Tool Rate

Cada uma mede um aspecto específico do comportamento.

Definição de Limites

Para transformar métricas em critérios objetivos, eu defino limites claros:

- Mínimo de 80% de acerto na seleção
- Máximo de 10% de chamadas desnecessárias
- Máximo de 15% de ferramentas erradas

Esses limites funcionam como critérios de aprovação.

Construção do Dataset

O dataset é composto por múltiplos cenários.

Cada cenário define:

- Contexto da execução
- Ferramenta esperada
- Argumentos esperados
- Ferramentas proibidas
- Justificativa da decisão

Essa estrutura garante que cada decisão seja avaliada com precisão.

Estrutura dos Cenários

Cada caso contém elementos essenciais:

- Identificador
- Entrada

- Etapa do loop
- Ferramenta esperada
- Argumentos esperados
- Ferramentas que não devem ser chamadas

Essa granularidade permite avaliação detalhada.

Execução da Eval Suite

A execução segue um fluxo estruturado:

- Carregamento dos contratos
- Execução de cada cenário
- Comparação entre esperado e obtido
- Cálculo das métricas

Ao final, um relatório consolidado é gerado.

Interpretação dos Resultados

Os resultados mostram onde o agente erra.

Por exemplo:

- Baixa seleção → problema no planner
- Argumentos incorretos → problema de contexto
- Chamadas desnecessárias → falta de controle
- Ferramentas erradas → ambiguidade nas skills

O Problema Real: Não é a LLM

Um dos aprendizados mais importantes desta etapa é:

O problema raramente está na LLM.

Na maioria dos casos, o problema está nos contratos.

Ambiguidade nas Skills

Quando duas skills possuem descrições semelhantes, a LLM não consegue diferenciar.

Por exemplo:

- Buscar logs estruturados
- Buscar logs históricos

Para humanos, a diferença é clara.

Para a LLM, não necessariamente.

Refinamento de Contratos

A solução é tornar as descrições mais específicas.

Por exemplo:

- Logs recentes via API nos últimos 60 minutos
- Logs históricos via banco de dados para análise temporal

Essa mudança simples pode aumentar drasticamente a acurácia.

Caso Real

Em um cenário prático, a acurácia estava em torno de 72%.

Após reescrever as descrições das skills:

- A acurácia subiu para 91%

Sem alterar modelo, prompt ou código.

Determinismo na Avaliação

Durante os testes, surgiu outro problema:

Resultados inconsistentes entre execuções.

Isso ocorre por conta da aleatoriedade da LLM.

Controle de Temperatura

Para resolver isso, eu utilizo temperatura igual a zero.

Isso faz com que:

- A LLM escolha sempre a opção mais provável
- Elimine variações aleatórias

Uso de Seed

Além da temperatura, eu utilizo seed fixa.

Isso garante:

- Reprodutibilidade
- Consistência entre execuções

Essa combinação é essencial para evals confiáveis.

Evolução do Dataset

Outro ajuste importante foi incluir o histórico de ferramentas utilizadas.

Isso permite avaliar decisões no contexto correto do loop.

Sem isso, a avaliação se torna imprecisa.

Ajustes no Planner

Também foi necessário ajustar o planner para evitar comportamentos indesejados.

Por exemplo:

Evitar que o agente continue coletando dados após já ter evidências suficientes.

Resultado Final

Após os ajustes:

- Tool Selection Accuracy atingiu 100%
- Argument Accuracy atingiu 100%
- Métricas passaram dentro dos limites definidos

Isso demonstra a eficácia do processo.

Comparação entre Arquiteturas

A eval também pode ser utilizada para comparar arquiteturas.

Observações importantes:

- ReAct tende a maior acurácia
- Reflection melhora decisões críticas
- Plan and Execute pode ser mais rígido

Isso confirma que arquitetura influencia decisão.

Ferramentas de Mercado

Implementar evals manualmente é possível, mas trabalhoso.

Por isso, existem ferramentas especializadas.

Um exemplo é:

- Promptfoo

Essa ferramenta permite:

- Definir datasets
- Configurar métricas
- Executar avaliações
- Gerar relatórios automáticos

Vantagens do Uso de Frameworks

Utilizar frameworks traz benefícios claros:

- Redução de esforço
- Padronização
- Melhor visualização
- Comparação entre modelos

Isso acelera o processo de validação.

A Grande Lição

A principal conclusão desta etapa é clara:

A qualidade do agente está nos contratos.

Não no modelo.

Não no framework.

Mas na forma como o problema é descrito.

Evolução do Sistema

Ao longo do módulo, eu evolui o agente em três dimensões:

Execução

Construção do runtime e contratos.

Cognição

Definição de arquiteturas cognitivas.

Integração

Conexão com sistemas reais.

Agora, adiciono a quarta dimensão:

Avaliação

Medição objetiva do comportamento.

Encerramento do Módulo

Ao final deste capítulo, eu completo o ciclo de construção de agentes.

Eu não apenas construo.

Eu:

- Defino comportamento
- Executo
- Observo
- Avalio
- Melhoro

Esse ciclo transforma desenvolvimento em engenharia.

Conclusão Final

O agente deixa de ser um experimento.

Ele passa a ser:

- Mensurável
- Confiável
- Evolutivo

E isso representa a maturidade necessária para levar agentes autônomos para produção.

Módulo 04: Memória e Evolução de Agentes Inteligentes

Capítulo 1: O Agente que lembra

Até este ponto, eu construí um agente completo.

Ele:

- Decide
- Executa
- Integra com sistemas reais
- Avalia resultados

No entanto, existe uma limitação crítica que impede esse agente de evoluir:

Ele não lembra de nada.

A cada execução, o agente começa do zero.

O Impacto da Falta de Memória

Considere um cenário recorrente.

O agente recebe um alerta de latência em um serviço.

Ele:

- Consulta métricas
- Analisa logs
- Verifica deploys
- Gera diagnóstico

No dia seguinte, o mesmo problema ocorre.

O agente repete todo o processo.

Ele não lembra:

- Da causa raiz anterior
- Da solução aplicada
- Das evidências coletadas

Isso gera:

- Reprocessamento desnecessário
- Maior custo
- Maior latência
- Perda de eficiência

Analogia com o Mundo Real

Eu gosto de pensar nisso como um profissional sem memória.

Imagine um médico que:

- Não lembra de consultas anteriores
- Não conhece o histórico do paciente
- Não registra diagnósticos

Esse profissional seria inviável.

E é exatamente assim que o agente opera sem memória.

A Solução: Introdução à Memória

Para resolver esse problema, eu introduzo um novo componente no agente:

Memória.

Mais especificamente, quatro tipos de memória.

Os Quatro Tipos de Memória

A memória do agente não é única. Ela é segmentada.

Memória Curta

Representa o estado atual da execução.

- Vive durante o ciclo
- Morre ao final da execução

Exemplo: latência atual medida durante a análise.

Memória Longa

Representa conhecimento persistente.

- Fatos sobre o sistema
- Configurações
- Preferências

Exemplo: o serviço de pagamentos utiliza PostgreSQL e possui um SLO definido.

Memória Episódica

Representa experiências passadas.

- Execuções anteriores
- Problemas resolvidos
- Lições aprendidas

Exemplo: um incidente anterior causado por deploy sem canário.

Memória Contextual

Representa conhecimento recuperado por similaridade.

- Baseada em embeddings
- Recupera informações relevantes automaticamente

Exemplo: ao investigar latência, o agente recupera episódios semelhantes.

Analogias para Fixação

Para facilitar o entendimento:

- Memória curta é a mesa de trabalho
- Memória longa é o caderno de referência
- Memória episódica é o diário profissional
- Memória contextual é um mecanismo de busca interno

Evolução do Contrato

Assim como ocorreu com skills e adapters, a memória é definida por contrato.

Cada tipo de memória possui:

- Tipo de implementação
- Políticas de uso
- Limites de armazenamento

O contrato governa o comportamento.

Configurações por Tipo

Cada memória possui características próprias:

- Limite de registros
- Regras de persistência
- Critérios de atualização

Por exemplo:

- Memória longa evita duplicação
- Memória episódica armazena resumos
- Memória contextual limita tokens

Políticas de Memória

A introdução de memória exige regras rigorosas.

A principal delas:

Nunca armazenar dados sensíveis.

Isso inclui:

- Tokens
- Senhas
- Credenciais

Essa regra não é opcional.

Validação de Fatos

Outro ponto crítico é a validação.

A memória longa deve armazenar apenas fatos confirmados.

Se uma informação não foi validada por ferramenta:

Ela não deve ser persistida.

Caso contrário, o agente passa a armazenar erros permanentemente.

Evolução do Agent Loop

Com memória, o Agent Loop evolui.

Antes:

- Perceber
- Planejar
- Executar
- Avaliar

Agora:

- Perceber
- Recuperar contexto
- Planejar
- Executar
- Avaliar
- Persistir memória

Recuperação de Contexto

Antes de decidir, o agente consulta suas memórias.

Ele busca:

- Episódios similares
- Fatos relevantes
- Contexto relacionado

Isso enriquece a decisão.

Persistência de Aprendizado

Após executar, o agente armazena:

- Fatos confirmados
- Resumo da execução
- Resultados obtidos

Esse processo transforma execução em aprendizado.

Implementação via Memory Adapter

Assim como adapters conectam o agente ao mundo externo, o Memory Adapter conecta o agente ao seu passado.

Ele é responsável por:

- Armazenar informações
- Recuperar contexto
- Atualizar registros

Tudo baseado nos contratos definidos.

Estrutura de Armazenamento

A memória é armazenada em uma estrutura dedicada.

Pode ser:

- Arquivos locais
- Banco de dados
- Sistemas de cache

A escolha é arquitetural.

Observabilidade da Memória

A memória também é rastreada.

O sistema registra:

- Quando algo foi armazenado
- Quando foi recuperado
- Qual informação foi utilizada

Isso permite auditoria completa.

Execução Sem Memória

Na primeira execução:

- O agente percorre todas as etapas
- Executa todas as ferramentas
- Constrói o diagnóstico do zero

Esse comportamento é esperado.

Execução Com Memória

Na segunda execução:

- O agente recupera contexto
- Identifica padrões
- Reduz etapas

O resultado é:

- Menos chamadas
- Menor latência
- Mesmo nível de qualidade

Impacto Prático

Com memória, o agente:

- Evita trabalho repetido
- Aprende com experiências
- Toma decisões mais rápidas

Isso não é apenas otimização.

É evolução comportamental.

Caso Real

Em um cenário de monitoramento:

- Antes: 5 etapas por incidente
- Depois: 2 a 3 etapas

Resultado:

- Redução de aproximadamente 37% nas chamadas
- Menor custo operacional
- Maior eficiência

Mudança de Paradigma

Sem memória:

O agente executa.

Com memória:

O agente aprende.

Essa é a diferença fundamental.

Comparação com Profissionais

Um profissional experiente não começa do zero.

Ele:

- Lembra de casos anteriores
- Reconhece padrões
- Aplica soluções conhecidas

O agente passa a operar da mesma forma.

Integração com Arquitetura Existente

A introdução de memória não quebra o sistema existente.

- Contratos continuam válidos
- Runtime permanece o mesmo
- Adapters continuam funcionando

Isso garante compatibilidade.

Ferramentas de Mercado

Existem soluções prontas para gerenciamento de memória.

Um exemplo é:

- Mem0

Essa ferramenta oferece:

- Persistência automática
- Gerenciamento de relevância
- Integração com frameworks

Abordagens Acadêmicas

Outro exemplo relevante é:

- MemGPT

Esse modelo trata memória como um sistema operacional interno.

Encerramento

Ao final desta etapa, eu transformo o agente.

Ele deixa de ser um executor stateless.

Ele passa a ser um sistema que:

- Lembra
- Aprende
- Evolui

Conclusão

A introdução de memória representa uma mudança fundamental.

O agente não apenas resolve problemas.

Ele acumula experiência.

E essa experiência reduz custo, melhora decisões e aumenta eficiência.

Capítulo 2: Embeddings Reflexao Evolutiva

Na etapa anterior, eu implementei memória longa e memória episódica.

Com isso, o agente passou a:

- Lembrar fatos
- Registrar experiências
- Reutilizar conhecimento

No entanto, surge uma limitação importante.

A recuperação de memória ainda depende de correspondência exata.

Isso significa que o agente só encontra aquilo que está literalmente igual ao que ele busca.

O Problema da Busca por Texto

Considere dois cenários:

- “Latência alta no checkout”
- “Tempo de resposta degradado”

Para um humano, ambos representam o mesmo problema.

Para o agente, utilizando busca por texto:

São completamente diferentes.

Esse é o limite da busca tradicional.

A Solução: Embeddings

Para resolver esse problema, eu introduzo embeddings.

Embeddings transformam texto em números.

Mais importante:

Transformam significado em representação numérica.

Como Funcionam os Embeddings

Cada texto é convertido em um vetor numérico.

Textos com significados semelhantes geram vetores próximos.

Por exemplo:

- Problemas de latência → vetores próximos
- Receitas culinárias → vetores distantes

Isso permite medir similaridade semântica.

Similaridade de Cosseno

Para comparar embeddings, utilizo similaridade de cosseno.

Essa métrica retorna um valor entre:

- 0 → completamente diferente
- 1 → idêntico

Um limite é definido no contrato.

Por exemplo:

- 0.7 → considerado relevante
- Abaixo disso → descartado

Impacto na Recuperação de Memória

Com embeddings, o agente deixa de buscar por palavras.

Ele passa a buscar por significado.

Isso permite:

- Encontrar informações relacionadas
- Recuperar contexto implícito
- Identificar padrões semânticos

Evolução da Memória Contextual

A memória contextual passa a utilizar embeddings.

Ela:

- Indexa informações
- Calcula similaridade
- Recupera fragmentos relevantes

Isso transforma completamente a capacidade do agente.

O Embed Adapter

Para suportar essa funcionalidade, eu implemento o Embed Adapter.

Ele possui três operações principais:

Indexar

Converte texto em embedding e armazena.

Buscar

Recebe uma consulta e retorna fragmentos similares.

Reindexar

Reconstrói o índice quando necessário.

Integração com o Sistema

O Embed Adapter é integrado ao Memory Adapter.

O fluxo passa a ser:

- Memória gera dados
- Embed Adapter indexa
- Consulta gera embedding
- Similaridade é calculada
- Resultados são retornados

Tudo governado por contrato.

Evolução do Agent Loop

O ciclo do agente evolui novamente.

Agora, antes do planejamento:

- O agente recupera contexto por similaridade

Esse contexto inclui:

- Fragmentos relevantes
- Episódios relacionados
- Conhecimento semântico

Enriquecimento do Planner

O Planner passa a receber um novo bloco:

Contexto enriquecido.

Esse bloco contém:

- Conhecimento relevante
- Experiências anteriores
- Fatos conhecidos
- Lições aprendidas

Novas Regras de Decisão

Com isso, o Planner passa a:

- Considerar contexto antes de decidir
- Evitar abordagens já falhas
- Aplicar lições anteriores
- Resolver conflitos de informação

A memória passa a influenciar diretamente a decisão.

Trade-off de Similaridade

O limite de similaridade é um ponto crítico.

Se for muito alto:

- Alta precisão
- Baixa cobertura

Se for muito baixo:

- Alta cobertura
- Baixa qualidade

O valor ideal depende do domínio.

Ajuste Baseado em Dados

Não existe valor universal.

O ajuste deve ser feito com base em:

- Métricas
- Evals
- Resultados observados

Esse ajuste é iterativo.

Limitações de Infraestrutura

O uso de embeddings exige armazenamento eficiente.

Para pequenos volumes:

- Arquivos locais são suficientes

Para grandes volumes:

- Bancos vetoriais são necessários

Um exemplo comum é o uso de extensões para bancos existentes.

Introdução à Reflexão Evolutiva

Até aqui, o agente:

- Lembra
- Recupera contexto

Mas ainda não aprende.

Memória não é aprendizado.

Diferença Fundamental

Memória armazena o passado.

Aprendizado transforma o passado em diretriz.

Essa é a função da reflexão evolutiva.

Evolução do Critique

Na unidade anterior, eu implementei o critique.

Ele avaliava cada execução.

Agora, ele evolui para reflection.

O Que é Reflection

Reflection é o processo de:

- Avaliar
- Extrair lições
- Persistir aprendizado

Esse aprendizado é reutilizado em execuções futuras.

Estrutura do Reflection

O Reflection possui três componentes:

Extração de Lições

Transforma execuções em aprendizados.

Detecção de Padrões

Identifica recorrência de comportamentos.

Injeção no Planner

Aplica aprendizado nas decisões futuras.

Estrutura das Lições

Cada lição contém:

- Situação
- Ação
- Resultado
- Aprendizado

Essa estrutura garante clareza.

CrITÉRIOS de Qualidade

Uma boa lição deve ser:

- Generalizável
- Acionável
- Baseada em evidência

Lições específicas demais são inúteis.

Lições genéricas demais também.

Detecção de Padrões

Padrões são identificados quando:

- Um comportamento se repete
- A frequência é significativa

Esses padrões geram ajustes estruturais.

Persistência de Aprendizado

As lições são armazenadas.

Elas passam a fazer parte da memória do agente.

Esse processo cria evolução contínua.

Injeção no Contexto

Durante o planejamento:

- Lições relevantes são recuperadas
- Inseridas no contexto
- Utilizadas na decisão

Isso fecha o ciclo de aprendizado.

Impacto no Comportamento

Com reflection, o agente:

- Evita erros recorrentes
- Melhora decisões
- Adapta comportamento

Ele deixa de repetir falhas.

Controle de Qualidade

Nem toda execução gera aprendizado.

Se não houver erro ou surpresa:

- Nenhuma lição é extraída

Isso evita poluição da memória.

Problema da Saturação

Excesso de lições prejudica o sistema.

Muitas lições:

- Confundem o planner
- Aumentam latência
- Reduzem qualidade

A solução é filtrar.

Políticas de Extração

As políticas definem:

- Quando extrair
- O que armazenar
- O que descartar

Essas regras são essenciais.

Caso Real

Em um cenário prático:

- Muitas lições foram armazenadas
- A maioria irrelevante

Após ajuste de políticas:

- Redução de ruído
- Melhoria de performance
- Decisões mais assertivas

Integração Completa

Neste ponto, o agente possui:

- Memória longa
- Memória episódica
- Memória contextual
- Reflection

Isso cria um ciclo completo:

- Executar
- Lembrar
- Buscar significado
- Aprender
- Evoluir

Encerramento

Ao final desta etapa, eu transformo o agente em um sistema adaptativo.

Ele não apenas reage.

Ele:

- Entende significado
- Aprende com experiência
- Melhora continuamente

Conclusão

A combinação de embeddings e reflection representa um salto de maturidade.

O agente deixa de ser reativo.

Ele passa a ser evolutivo.

E isso define a diferença entre um sistema funcional e um sistema inteligente.

Capítulo 3: Evals Memoria Fechamento

Nas últimas aulas, eu introduzi a memória no agente. Estructurei quatro tipos distintos de memória, implementei mecanismos de busca por significado,

reflexão evolutiva e um ciclo onde o agente passa a lembrar, recuperar e aprender com cada execução.

Até aqui, tudo parece bastante promissor. O agente agora possui histórico, contexto acumulado e capacidade de evolução. No entanto, existe uma pergunta fundamental que precisa ser respondida com rigor técnico: essa memória realmente está funcionando?

A presença de memória em um agente não é automaticamente benéfica. Na prática, ela pode tanto melhorar quanto prejudicar a tomada de decisão. Por isso, antes de assumir qualquer ganho, eu preciso avaliar.

Cenários de impacto da memória

Para entender o comportamento da memória, eu considero três cenários possíveis.

No primeiro cenário, a memória é útil. O agente investiga um problema de latência e recupera da memória que, em um caso anterior, a causa foi um deploy recente. Com isso, ele prioriza essa hipótese e encontra rapidamente a causa raiz. O processo que antes levaria cinco etapas passa a ser resolvido em três. Aqui, a memória claramente melhora a decisão.

No segundo cenário, a memória é irrelevante. O agente investiga um erro 500 e recupera múltiplos fragmentos de memória que não têm relação com o problema atual. O Planner precisa processar esses dados, consumindo tokens, mas no final toma exatamente a mesma decisão que tomaria sem memória. Nesse caso, a memória apenas gera custo, sem benefício.

No terceiro cenário, a memória é perigosa. O agente investiga um timeout e recupera da memória que um determinado serviço utiliza um limite de 30 segundos. No entanto, esse valor foi alterado recentemente para 10 segundos. O agente confia em uma informação desatualizada e produz um diagnóstico incorreto. Aqui, a memória introduz erro, o que é ainda mais grave do que não ajudar.

Esses três cenários deixam claro que memória não é garantia de melhoria. Ela pode otimizar, ser neutra ou causar falhas.

A necessidade de avaliação

Diante disso, a única forma confiável de entender o impacto da memória é medir. Eu preciso avaliar de forma estruturada.

Essa abordagem não é nova. Na unidade anterior, eu já construí benchmarks evolutivos para comparar arquiteturas cognitivas. Trabalhei com datasets, métricas, limites e relatórios comparativos. Agora, aplico exatamente a mesma lógica, mas com um foco diferente: medir o impacto da memória.

O método permanece o mesmo. O que muda é o objeto da avaliação.

Métricas de impacto da memória

Para realizar essa avaliação, eu defino seis métricas principais.

A primeira é o Retrieval Precision. Ela mede a proporção de fragmentos recuperados que são realmente úteis. Se o agente recupera cinco fragmentos e apenas dois são relevantes, a precisão é de 40%. Essa métrica indica o nível de ruído no contexto.

A segunda é o Retrieval Recall. Ela responde se o agente conseguiu recuperar tudo o que era importante. Se existiam quatro fragmentos relevantes e apenas dois foram recuperados, o recall é de 50%. Aqui eu avalio a cobertura da recuperação.

A terceira métrica é o Memory Utilization. Não basta recuperar o contexto, o agente precisa utilizá-lo. Se o Planner recebe cinco fatos e utiliza apenas um, a utilização é de 20%. Essa métrica mede o aproveitamento do contexto.

A quarta métrica é o Hallucination from Memory. Ela identifica quando o agente inventa ou altera dados em relação à memória. Se a memória indica uma versão específica de um sistema e o agente responde com outra, há alucinação.

A quinta métrica é o Decision Improvement. Essa é a métrica central. Eu comparo a mesma execução com e sem memória. Se com memória o agente resolve o problema em menos etapas ou com maior precisão, o impacto é positivo.

A sexta métrica é o Lesson Quality. Ela avalia se as lições extraídas são úteis, generalizáveis e reutilizáveis em execuções futuras.

Entre todas, a métrica mais importante é o Decision Improvement. Ela responde diretamente à pergunta principal: a memória está ajudando ou atrapalhando?

Dataset de avaliação

Para medir essas métricas, eu utilizo um dataset estruturado. Cada caso contém:

Um identificador

Uma entrada representando o problema

O contexto de memória disponível

Fatos relevantes

Episódios relevantes

Lições relevantes

A decisão esperada

O resultado esperado

Esse formato permite avaliar o comportamento do agente em diferentes cenários. Mais importante ainda, permite executar o mesmo caso com e sem memória, o que viabiliza o cálculo do Decision Improvement.

Execução dos evals

Ao rodar o processo de avaliação, cada caso é executado duas vezes. Primeiro sem memória, simulando um cenário baseline. Depois com memória, utilizando o contexto recuperado.

Os resultados são comparados linha a linha. Ao final, um relatório consolida as métricas, indicando quais passaram ou falharam em relação aos limites definidos.

É importante entender que os resultados podem variar entre execuções. Modelos de linguagem não são determinísticos. Uma única execução não é suficiente para concluir nada. A avaliação precisa ser estatística, baseada em múltiplas execuções e análise de tendência.

Interpretação dos resultados

Ao analisar o relatório, eu observo cada métrica individualmente.

Se o Retrieval Precision está baixo, isso indica que há ruído no contexto. A ação correta é aumentar o limite de similaridade na recuperação.

Se o Retrieval Recall está baixo, o agente está deixando de recuperar informações importantes. Nesse caso, eu ajusto os critérios de busca ou melhora a qualidade dos fragmentos.

Se o Memory Utilization é baixo, o problema está no Planner, que não está considerando o contexto. É necessário revisar as regras de decisão.

Se há alta incidência de Hallucination from Memory, isso indica dados desatualizados ou inconsistentes. A solução é implementar políticas de expiração e revalidação da memória.

Se o Decision Improvement é negativo, a memória está prejudicando o agente. Aqui, eu analiso as demais métricas para identificar a causa raiz.

Se o Lesson Quality é baixo, as lições estão mal formuladas. É necessário ajustar o processo de reflexão.

Cada métrica aponta diretamente para um tipo de ajuste. O problema não está no agente em si, mas nos parâmetros e políticas que governam seu comportamento.

Caso real de falha de memória

Um exemplo prático ilustra bem esse processo. Um agente de performance apresentava um ganho médio significativo com o uso de memória. No entanto, a avaliação revelou uma queda de desempenho em um serviço específico.

Ao investigar, identifiquei que o agente estava utilizando informações desatualizadas sobre a versão de um sistema de cache. A infraestrutura havia sido atualizada recentemente, mas a memória ainda refletia o estado antigo.

Isso levou a diagnósticos incorretos. A solução foi implementar uma política de expiração de memória, onde dados antigos precisam ser revalidados.

Sem o processo de avaliação, esse problema teria passado despercebido.

Ferramentas para avaliação em escala

Embora seja possível implementar todo o processo manualmente, existem ferramentas especializadas para esse tipo de avaliação.

Uma delas é o framework Ragas.

Ele oferece métricas prontas, como precisão de contexto, recall, fidelidade e uso do contexto. Além disso, facilita a execução em larga escala, com suporte a grandes datasets e geração automática de relatórios.

Para fins educacionais, a implementação manual é importante para entender o funcionamento interno. Em produção, o uso de frameworks como esse reduz significativamente o esforço.

Memória como arquitetura

A principal lição desta unidade é que memória não deve ser tratada como uma funcionalidade opcional. Ela é parte da arquitetura do agente.

A introdução de memória altera o ciclo de execução. Novas etapas são adicionadas, como recuperação de contexto e persistência. O Planner passa a operar com informações enriquecidas. Regras e políticas passam a governar o uso da memória.

Tratar memória como um simples complemento leva a soluções frágeis. É necessário projetar, governar, medir e ajustar continuamente.

Visão geral do sistema

Ao longo do módulo, construí uma arquitetura completa de agentes.

Comecei com contratos bem definidos. Evoluí para arquiteturas cognitivas. Integrei com cenários reais. E agora adicionei memória, reflexão e avaliação.

O resultado é um agente que aprende com a própria execução.

Utilizando uma analogia, o modelo é o motor, as regras são os freios, o Planner é a direção, as habilidades são o combustível, a memória é o GPS com histórico e a reflexão é o mecanismo de aprendizado contínuo.

Encerramento

Neste ponto, o agente não apenas executa tarefas, mas evolui com o tempo. Ele passa a atuar de forma mais autônoma, utilizando experiências passadas para melhorar decisões futuras.

O diferencial não está no modelo utilizado, mas na arquitetura construída ao redor dele. Contratos, adapters, memória, reflexão e avaliação formam um sistema coeso.

Esse é o verdadeiro fundamento da engenharia aplicada a agentes inteligentes.

